



분류-Kaggle 필사

1조 김예나, 유연, 장인서

목차

#01 캐글 산탄데르 고객 만족 예측

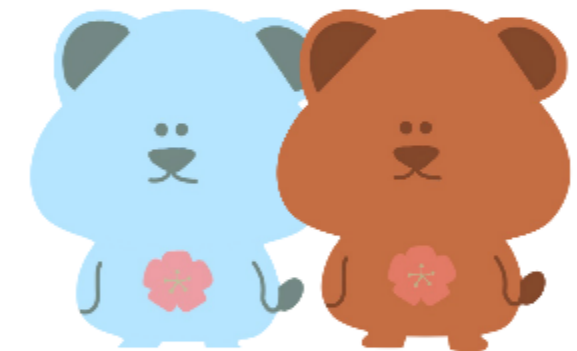
#02 캐글 신용카드 사기 검출

#03 Beginner Friendly CatBoost with Optuna



캐글 산탄데르 고객 만족 예측

Decision Tree And Random Forest Classifier Models



#01 캐글 산탄데르 고객 만족 예측

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings

warnings.filterwarnings('ignore')
cust_df = pd.read_csv('./train_santander.csv', encoding='latin-1')
print('dataset shape:', cust_df.shape)
cust_df.head(3)
```

dataset shape: (76020, 371)

	ID	var3	var15	imp_ent_var16_ult1	imp_op_var39_comer_ult1	imp_op_var39_comer_ult3	imp_op_var40_comer_ult1	imp_op_var40_comer_ult3	imp_op_var40_efect_ult1	imp_op_var40_efect_ult3
0	1	2	23	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	3	2	34	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	4	2	23	0.0	0.0	0.0	0.0	0.0	0.0	0.0

3 rows × 371 columns

- train_Santander.csv 파일을 새로운 노트북이 생성된 디렉터리로 옮김
- XGBoost는 사이킷런 래퍼 사용
- 필요한 모듈 로딩, 학습 데이터 DataFrame으로 로딩
- 클래스 값 칼럼 포함한 feature 371개 존재

```
cust_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 76020 entries, 0 to 76019
Columns: 371 entries, ID to TARGET
dtypes: float64(111), int64(260)
memory usage: 215.2 MB
```

- Feature 타입과 Null 값 알아보기
- 111개의 featur가 float형, 260개의 feature가 int형
=>모든 feature 숫자형, Null 값 없음

#01 캐글 산탄데르 고객 만족 예측

```
print(cust_df['TARGET'].value_counts())
unsatisfied_cnt = cust_df[cust_df['TARGET'] == 1].TARGET.count()
total_cnt = cust_df.TARGET.count()
print('unsatisfied 비율은 {0:.2f}'.format((unsatisfied_cnt / total_cnt)))
```

- Target 속성의 값의 분포
- 불만족 4%

TARGET

0 73012

1 3008

cust_df.describe()

	ID	var3	var15	imp_est_var16_ult1	imp_op_var39_comer_ult1	imp_op_var39_comer_ult3	imp_op_var40_comer_ult1	imp_op_var40_comer_ult3	imp_op_var40_efect_ul
count	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000
mean	75964.050723	-1523.199277	33.212865	86.208265	72.363067	119.529632	3.559130	6.472698	0.4125
std	43781.947379	39033.462364	12.956486	1614.757313	339.315831	546.266294	93.155749	153.737066	30.6042
min	1.000000	-999999.000000	5.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
25%	38104.750000	2.000000	23.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
50%	76043.000000	2.000000	28.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
75%	113748.750000	2.000000	40.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0000
max	151838.000000	238.000000	105.000000	210000.000000	12888.030000	21024.810000	8237.820000	11073.570000	6600.0000

1 rows x 371 columns

- var3 칼럼의 경우 min 값이 -999999
- => 2로 변환
- ID feature는 단순 식별자 -> feature 드롭

```
cust_df['var3'].replace(-999999, 2, inplace=True)
cust_df.drop('ID', axis=1, inplace=True)
```

```
# 피쳐 세트와 레이블 세트 분리. 레이블 컬럼은 DataFrame의 맨 마지막에 위치해 컬럼 위치 -1로 분리
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
print('피쳐 데이터 shape:{0}'.format(X_features.shape))
```

- 클래스 데이터 세트와 feature 데이터 세트 분리
- 별도의 데이터 세트로 저장

#01 캐글 산탄데르 고객 만족 예측

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels,
                                                    test_size=0.2, random_state=0)

train_cnt = y_train.count()
test_cnt = y_test.count()
print('학습 세트 Shape:{0}, 테스트 세트 Shape:{1}'.format(X_train.shape , X_test.shape))

print(' 학습 세트 레이블 값 분포 비율')
print(y_train.value_counts()/train_cnt)
print('\n 테스트 세트 레이블 값 분포 비율')
print(y_test.value_counts()/test_cnt)
```

- 학습 성능 평가 위해 데이터 세트 분리
- 비대칭한 데이터 세트
- => Target 값 분포도가 비슷하게 추출됐는지 확인
- 모두 원본 데이터와 유사하게 4% 정도의 값1

```
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
                                             test_size=0.3, random_state=0)
```

- 학습 검증 데이터 세트로 만들기

#01 캐글 산탄데르 고객 만족 예측

XGBoost 모델 학습과 하이퍼 파라미터 튜닝

```
import xgboost as xgb
from sklearn.metrics import roc_auc_score

# DMatrix로 변환
dtrain = xgb.DMatrix(X_tr, label=y_tr)
dval = xgb.DMatrix(X_val, label=y_val)
dtest = xgb.DMatrix(X_test)

# 하이퍼파라미터 설정
params = {
    'objective': 'binary:logistic', # 이진 분류
    'eval_metric': 'auc',
    'learning_rate': 0.05,
    'max_depth': 3,
    'seed': 156
}

# 평가용 watchlist 지정
wlist = [(dtrain, 'train'), (dval, 'eval')]

# 학습 수행 (조기 종료 포함)
xgb_model = xgb.train(params=params,
                      dtrain=dtrain,
                      num_boost_round=500,
                      evals=wlist,
                      early_stopping_rounds=100,
                      verbose_eval=True)

# 테스트 데이터 예측 및 평가
pred_probs = xgb_model.predict(dtest)
roc_score = roc_auc_score(y_test, pred_probs)
print('ROC AUC: {:.4f}'.format(roc_score))

[302] train-auc:0.86758 eval-auc:0.83547
[303] train-auc:0.86766 eval-auc:0.83537
[304] train-auc:0.86768 eval-auc:0.83536
```

- 우회하기 위해서 XGBoost의 저수준 API인 `xgboost.train()`
- 하이퍼 파라미터는 `params` 딕셔너리
- `n_estimators`는 500, `early_stopping_rounds`를 100
- 테스트 데이터 세트로 예측 시 ROC AUC는 약 0.8429

#01 캐글 산탄데르 고객 만족 예측

```
from hyperopt import hp

xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 15, 1),
                    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)}
}
```

- HyperOpt 이용, 베이지안 최적화 기반 XGBoost 하이퍼 파라미터 수행

- 목적 함수 만들기
 - 3 Fold 교차 검증 이용
 - 평균 ROC-AUC 값 반환, -1 곱함
 - Estimators는 0으로 줄임
- Early_stopping_rounds 30으로 줄임, 튜닝 후 증가

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score
from xgboost import XGBClassifier
import numpy as np

def objective_func(search_space):
    xgb_clf = XGBClassifier(
        n_estimators=100,
        max_depth=int(search_space['max_depth']),
        min_child_weight=int(search_space['min_child_weight']),
        colsample_bytree=search_space['colsample_bytree'],
        learning_rate=search_space['learning_rate'],
        early_stopping_rounds=30,
        eval_metric='auc',
        random_state=156
    )

    roc_auc_list = []
    kf = KFold(n_splits=3)

    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        xgb_clf.fit(
            X_tr, y_tr,
            eval_set=[(X_val, y_val)],
            verbose=False
        )

        score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[:, 1])
        roc_auc_list.append(score)

    return -1 * np.mean(roc_auc_list)
```


#01 캐글 산탄데르 고객 만족 예측

```
from hyperopt import hp, fmin, tpe, Trials
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score
import xgboost as xgb
import numpy as np

# 1. 탐색 공간 정의
xgb_search_space = {
    'max_depth': hp.quniform('max_depth', 5, 15, 1),
    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
}

# 2. 목적 함수 정의 (xgboost.train() 사용)
def objective_func(search_space):
    params = {
        'objective': 'binary:logistic',
        'eval_metric': 'auc',
        'max_depth': int(search_space['max_depth']),
        'min_child_weight': int(search_space['min_child_weight']),
        'colsample_bytree': search_space['colsample_bytree'],
        'learning_rate': search_space['learning_rate'],
        'seed': 0
    }

    roc_auc_list = []
    kf = KFold(n_splits=3, shuffle=True, random_state=0)

    for tr_index, val_index in kf.split(X_train):
        X_tr, X_val = X_train.iloc[tr_index], X_train.iloc[val_index]
        y_tr, y_val = y_train.iloc[tr_index], y_train.iloc[val_index]

        dtrain = xgb.DMatrix(X_tr, label=y_tr)
        dval = xgb.DMatrix(X_val, label=y_val)

        model = xgb.train(
            params,
            dtrain,
            num_boost_round=100,
            evals=[(dval, 'eval')],
            early_stopping_rounds=30,
            verbose_eval=False
        )

        preds = model.predict(dval)
        auc = roc_auc_score(y_val, preds)
        roc_auc_list.append(auc)

    return -1 * np.mean(roc_auc_list)

# 3. 하이퍼파라미터 튜닝 실행
trials = Trials()

best = fmin(
    fn=objective_func,
    space=xgb_search_space,
    algo=tpe.suggest,
    max_evals=50,
    trials=trials,
    rstate=np.random.default_rng(seed=30)
)

# 4. 결과 출력
print("best:", best)
```

100%|██████████| 50/50 [17:20<00:00, 20.81s/trial, best loss: -0.83943932807024]
best: {'colsample_bytree': np.float64(0.5829020839311067), 'learning_rate': np.float64(0.10548643039738301), 'max_depth': np.float64(5.0), 'min_child_weight': np.float64(6.0)}

- Fmin() 함수 호출, max_eval=50회만큼 반복, 최적 파라미터 도출
- 50회만큼 교차 검증 반복

#01 캐글 산탄데르 고객 만족 예측

```
params = {
    'objective': 'binary:logistic',
    'eval_metric': 'auc',
    'learning_rate': round(best['learning_rate'], 5),
    'max_depth': int(best['max_depth']),
    'min_child_weight': int(best['min_child_weight']),
    'colsample_bytree': round(best['colsample_bytree'], 5),
    'seed': 42
}
```

DMatrix 변환

```
dtrain = xgb.DMatrix(X_tr, label=y_tr)
dval = xgb.DMatrix(X_val, label=y_val)
dtest = xgb.DMatrix(X_test)
```

학습

```
model = xgb.train(
    params=params,
    dtrain=dtrain,
    num_boost_round=500,
    evals=[(dval, 'eval')],
    early_stopping_rounds=100,
    verbose_eval=True
)
```

예측 및 평가

```
preds = model.predict(dtest)
xgb_roc_score = roc_auc_score(y_test, preds)
print('ROC AUC: {:.4f}'.format(xgb_roc_score))
```

```
[131] eval-auc:0.83617
[132] eval-auc:0.83616
[133] eval-auc:0.83611
[134] eval-auc:0.83592
[135] eval-auc:0.83590
[136] eval-auc:0.83600
[137] eval-auc:0.83598
[138] eval-auc:0.83596
```

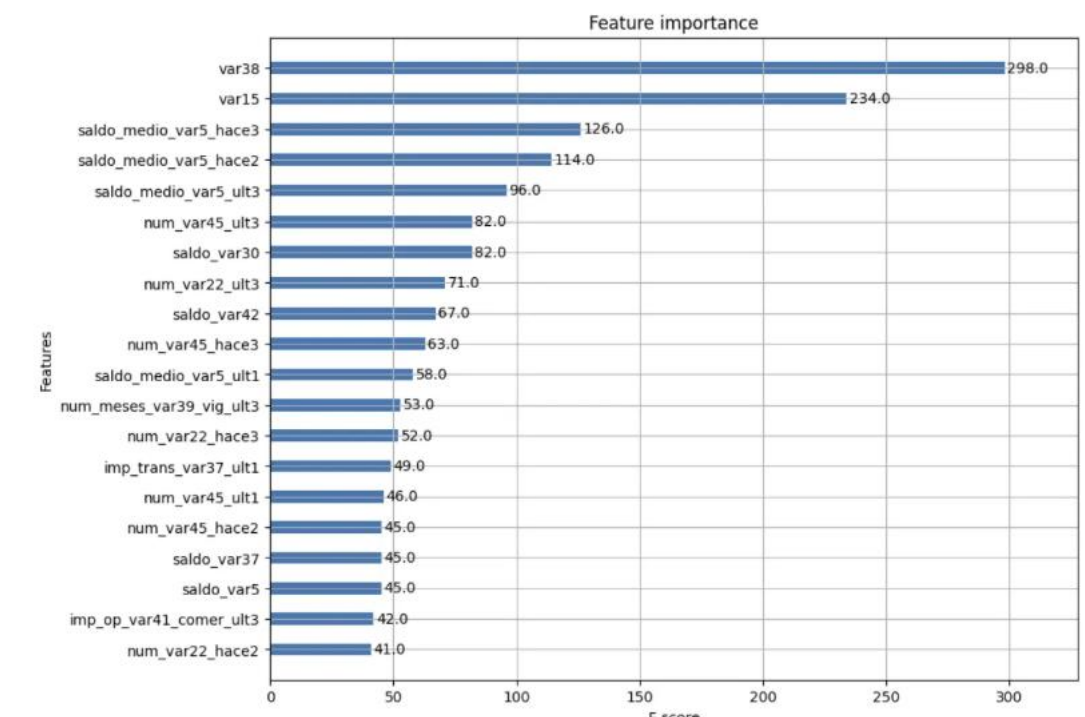
- 튜닝 이후 미세하게 개선
- 수행 시간 오래 걸림

- 튜닝된 모델에서 feature 중요도 그래프로 나타내기
 - Plot_importance() 메서드 이용

```
# 기존에 model로 학습했던 Booster를 xgb_clf라는 이름으로 저장
xgb_clf = model
```

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline
```

```
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
plot_importance(xgb_clf, ax=ax, max_num_features=20, height=0.4)
plt.show()
```



#01 캐글 산탄데르 고객 만족 예측

LightGBM 모델 학습과 하이퍼 파라미터 튜닝

```
from lightgbm import early_stopping, log_evaluation

lgbm_clf = LGBMClassifier(n_estimators=500)

eval_set = [(X_val, y_val)]

lgbm_clf.fit(
    X_tr, y_tr,
    eval_set=eval_set,
    eval_metric="auc",
    callbacks=[
        early_stopping(stopping_rounds=100),
        log_evaluation(period=10) # 10번째마다 로그 출력
    ]
)
```

```
[LightGBM] [Info] Number of positive: 1658, number of negative: 40913
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.1260s
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 13308
[LightGBM] [Info] Number of data points in the train set: 42571, number of used features: 24;
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.038947 -> initscore=-3.205836
[LightGBM] [Info] Start training from score -3.205836
Training until validation scores don't improve for 100 rounds
[10]  valid_0's auc: 0.826285 valid_0's binary_logloss: 0.139756
[20]  valid_0's auc: 0.829603 valid_0's binary_logloss: 0.136324
[30]  valid_0's auc: 0.83085  valid_0's binary_logloss: 0.135507
[40]  valid_0's auc: 0.831749 valid_0's binary_logloss: 0.135283
[50]  valid_0's auc: 0.831734 valid_0's binary_logloss: 0.13536
[60]  valid_0's auc: 0.8319   valid_0's binary_logloss: 0.135368
[70]  valid_0's auc: 0.831659 valid_0's binary_logloss: 0.135611
[80]  valid_0's auc: 0.831212 valid_0's binary_logloss: 0.135846
[90]  valid_0's auc: 0.830845 valid_0's binary_logloss: 0.136074
[100] valid_0's auc: 0.830154 valid_0's binary_logloss: 0.136377
[110] valid_0's auc: 0.830052 valid_0's binary_logloss: 0.136566
[120] valid_0's auc: 0.829622 valid_0's binary_logloss: 0.136997
[130] valid_0's auc: 0.829409 valid_0's binary_logloss: 0.137231
[140] valid_0's auc: 0.829175 valid_0's binary_logloss: 0.137451
Early stopping, best iteration is:
```

XGBoost 학습

```
XGBClassifier.fit() +
early_stopping_rounds
```

```
xgb.train() + DMatrix
```

조기 종료 오류

```
unexpected keyword argument
```

```
early_stopping_rounds
```

 안정 작동

변수명 충돌

```
xgb_clf 가 학습 안 됨 → 오류
```

```
xgb_clf = model
```

 로 해결

LightGBM 조기 종료

```
early_stopping_rounds=100 → 오류
```

```
callbacks=
[early_stopping(100)]
```

로그 출력

```
verbose
```

 사용 불가

```
log_evaluation()
```

 사용

- 테스트 데이터 세트로 평가된 ROC-AUC 값 확인
- LightGBM 직접 수행 -> 시간 단축

#01 캐글 산탄데르 고객 만족 예측

```
lgbm_search_space = {'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
                     'max_depth': hp.quniform('max_depth', 100, 160, 1),
                     'min_child_samples': hp.quniform('min_child_samples', 60, 100, 1),
                     'subsample': hp.uniform('subsample', 0.7, 1),
                     'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
                     }
```

- HyperOpt 이용, 튜닝 수행
- 하이퍼 파라미터 검색 공간 설정

- 목적 함수 생성
- XGBoost와 코드 비슷
- LGBMClassifier 객체 생성 부분만 달라짐

```
from lightgbm import early_stopping

def objective_func(search_space):
    lgbm_clf = LGBMClassifier(n_estimators=100,
                              num_leaves=int(search_space['num_leaves']),
                              max_depth=int(search_space['max_depth']),
                              min_child_samples=int(search_space['min_child_samples']),
                              subsample=search_space['subsample'],
                              learning_rate=search_space['learning_rate'])

    roc_auc_list = []
    kf = KFold(n_splits=3)

    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        # 조기 종료를 콜백으로 전달
        lgbm_clf.fit(
            X_tr, y_tr,
            eval_set=[(X_val, y_val)],
            eval_metric="auc",
            callbacks=[early_stopping(stopping_rounds=30)]
        )

        score = roc_auc_score(y_val, lgbm_clf.predict_proba(X_val)[:, 1])
        roc_auc_list.append(score)

    return -1 * np.mean(roc_auc_list)
```

#01 캐글 산탄데르 고객 만족 예측

```
from hyperopt import fmin, tpe, Trials

trials = Trials()

# fmin() 함수를 호출. max_evals 지정된 횟수만큼 반복 후 목적함수의 최소값을 가지는 최적 입력값
best = fmin(fn=objective_func, space=lgbm_search_space, algo=tpe.suggest,
           max_evals=50, # 최대 반복 횟수를 지정합니다.
           trials=trials, rstate=np.random.default_rng(seed=30))

print('best:', best)
```

[LightGBM] [Info] Number of positive: 1609, number of negative: 38935
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.08
You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.
[LightGBM] [Info] Total Bins 13094
[LightGBM] [Info] Number of data points in the train set: 40544, number of used features:
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.039685 -> initscore=-3.186281
[LightGBM] [Info] Start training from score -3.186281
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[17] valid_0's auc: 0.835208 valid_0's binary_logloss: 0.133511
[LightGBM] [Info] Number of positive: 1560, number of negative: 38984
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.10
You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.
[LightGBM] [Info] Total Bins 12950
[LightGBM] [Info] Number of data points in the train set: 40544, number of used features:
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.038477 -> initscore=-3.218465
[LightGBM] [Info] Start training from score -3.218465
Training until validation scores don't improve for 30 rounds
Early stopping, best iteration is:
[43] valid_0's auc: 0.835286 valid_0's binary_logloss: 0.136897
[LightGBM] [Info] Number of positive: 1579, number of negative: 38965
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.08
You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.
[LightGBM] [Info] Total Bins 12922
[LightGBM] [Info] Number of data points in the train set: 40544, number of used features:

100%|██████████| 50/50 [06:16<00:00, 7.53s/trial, best loss: -0.8362219933743916]
best: {'learning_rate': np.float64(0.04589753644367517), 'max_depth': np.float64(119.0),

- fmin() 호출하여 최적 파라미터 도출

```
from lightgbm import early_stopping, log_evaluation

lgbm_clf = LGBMClassifier(
    n_estimators=500,
    num_leaves=int(best['num_leaves']),
    max_depth=int(best['max_depth']),
    min_child_samples=int(best['min_child_samples']),
    subsample=round(best['subsample'], 5),
    learning_rate=round(best['learning_rate'], 5)
)

# 학습 (조기 종료 콜백 사용)
lgbm_clf.fit(
    X_tr, y_tr,
    eval_set=[(X_val, y_val)],
    eval_metric="auc",
    callbacks=[
        early_stopping(stopping_rounds=100),
        log_evaluation(period=10)
    ]
)

# 테스트 데이터에 대한 ROC AUC 평가
lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[:, 1])
print('ROC AUC: {:.4f}'.format(lgbm_roc_score))
```

ROC AUC: 0.8412

- LightGBM 학습 후 테스트 데이터 세트에서 ROC-AUC 평가
- 마찬가지로 callbacks 형태로 early_stopping_rounds 수행

#02 Decision Tree And Random Forest Classifier Models

-1. 데이터 검토

```
# I read the dataset with the pandas library.
from google.colab import files
uploaded = files.upload()
data = pd.read_csv("drug200.csv")
```

data						
	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	F	LOW	HIGH	11.567	drugC
196	16	M	LOW	HIGH	12.006	drugC
197	52	M	NORMAL	HIGH	9.894	drugX
198	23	M	NORMAL	NORMAL	14.020	drugX
199	40	F	LOW	NORMAL	11.349	drugX

200 rows × 6 columns

#02 Decision Tree And Random Forest Classifier Models

-1. 데이터 검토

```
# information of data
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Age              200 non-null    int64
1   Sex              200 non-null    object
2   BP               200 non-null    object
3   Cholesterol      200 non-null    object
4   Na_to_K          200 non-null    float64
5   Drug             200 non-null    object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```

```
# Dataset Values(Max, min, std..)
data.describe()
```

	Age	Na_to_K
count	200.000000	200.000000
mean	44.315000	16.084485
std	16.544315	7.223956
min	15.000000	6.269000
25%	31.000000	10.445500
50%	45.000000	13.936500
75%	58.000000	19.380000
max	74.000000	38.247000

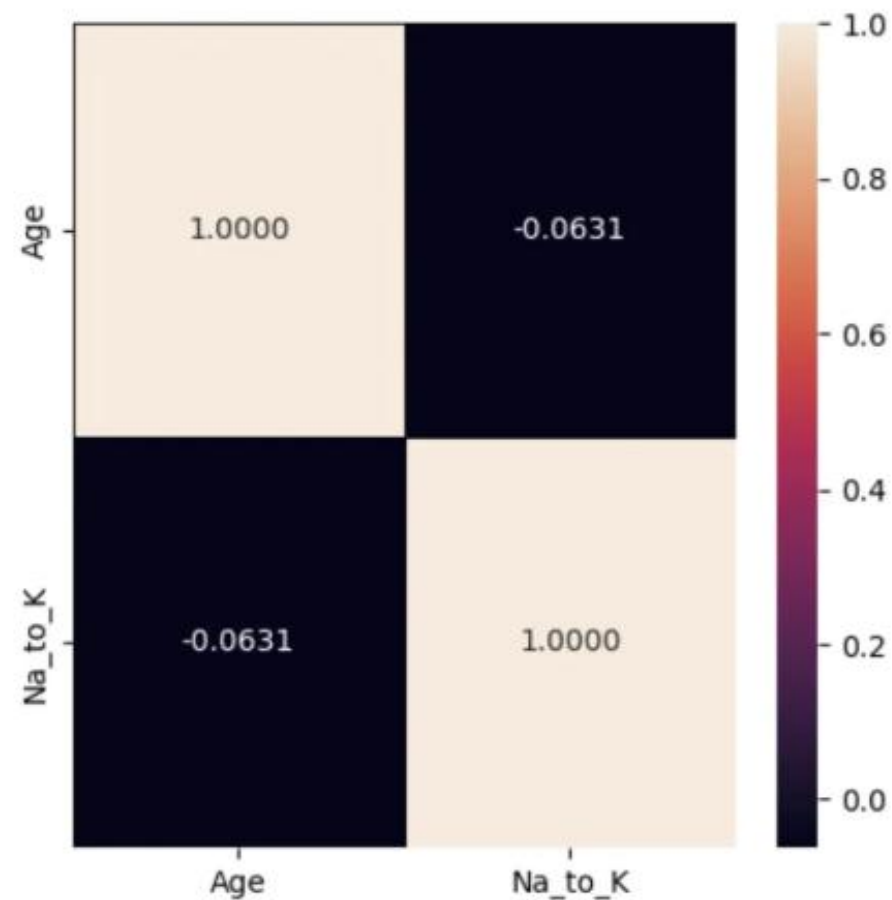
```
# Data Correlation: Relationship between columns
data.corr(numeric_only = True)
```

	Age	Na_to_K
Age	1.000000	-0.063119
Na_to_K	-0.063119	1.000000

#02 Decision Tree And Random Forest Classifier Models

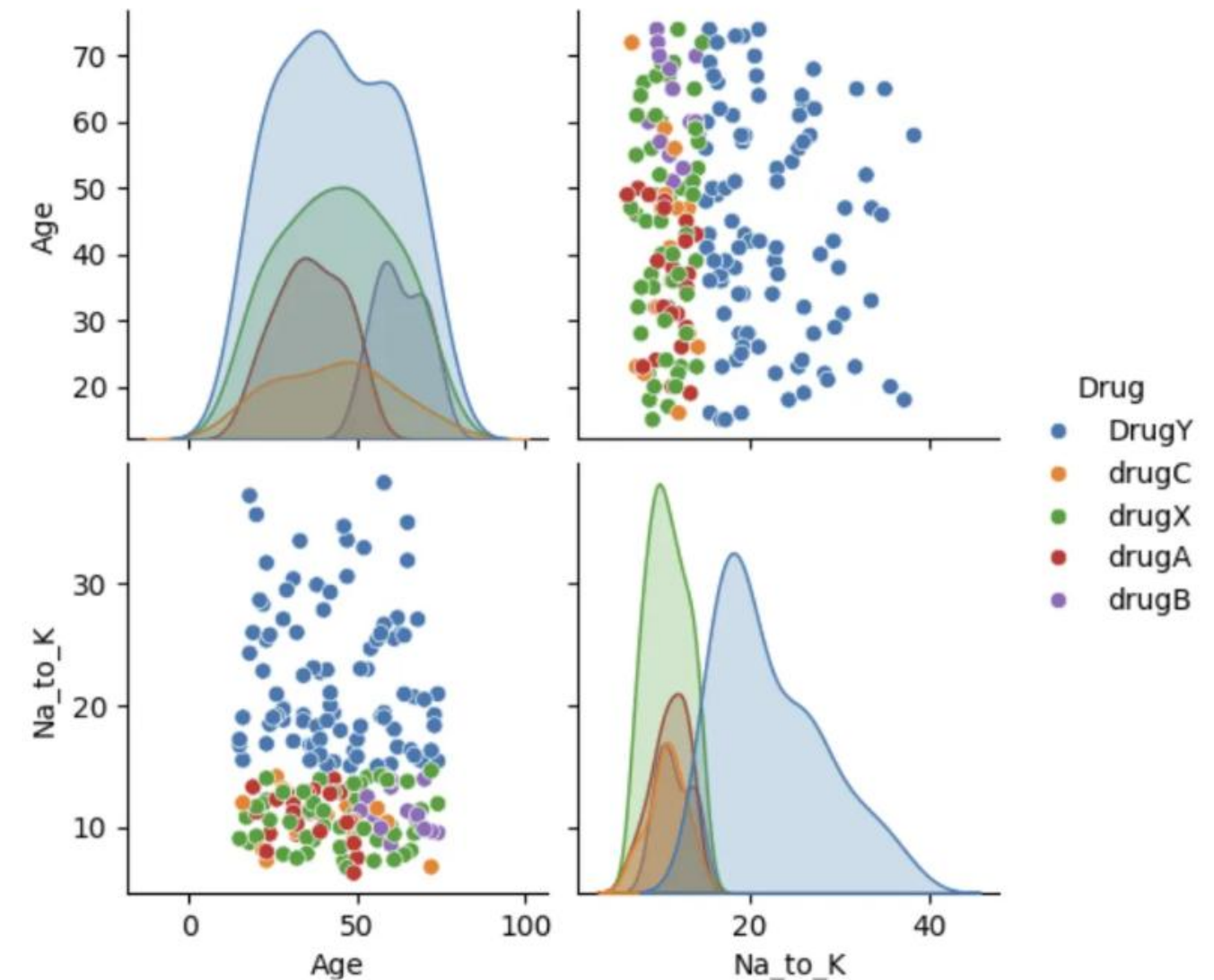
-1. 데이터 검토

```
f, ax = plt.subplots(figsize = (5,5))
sns.heatmap(data.corr(numeric_only = True), annot = True, linewidths=0.5, linecolor = "black", fmt = ".4f",
plt.show()
```



```
# Visualization of correlation results with seaborn library pairplot
sns.pairplot(data, hue = "Drug")
```

<seaborn.axisgrid.PairGrid at 0x7fb7ae19a050>



#02 Decision Tree And Random Forest Classifier Models

-1. 데이터 검토

```
# Data in the Drug column
data["Drug"].value_counts()
```

count	
Drug	
DrugY	91
drugX	54
drugA	23
drugC	16
drugB	16

dtype: int64

```
# Data in the age column
data["Age"].value_counts(dropna=False)
```

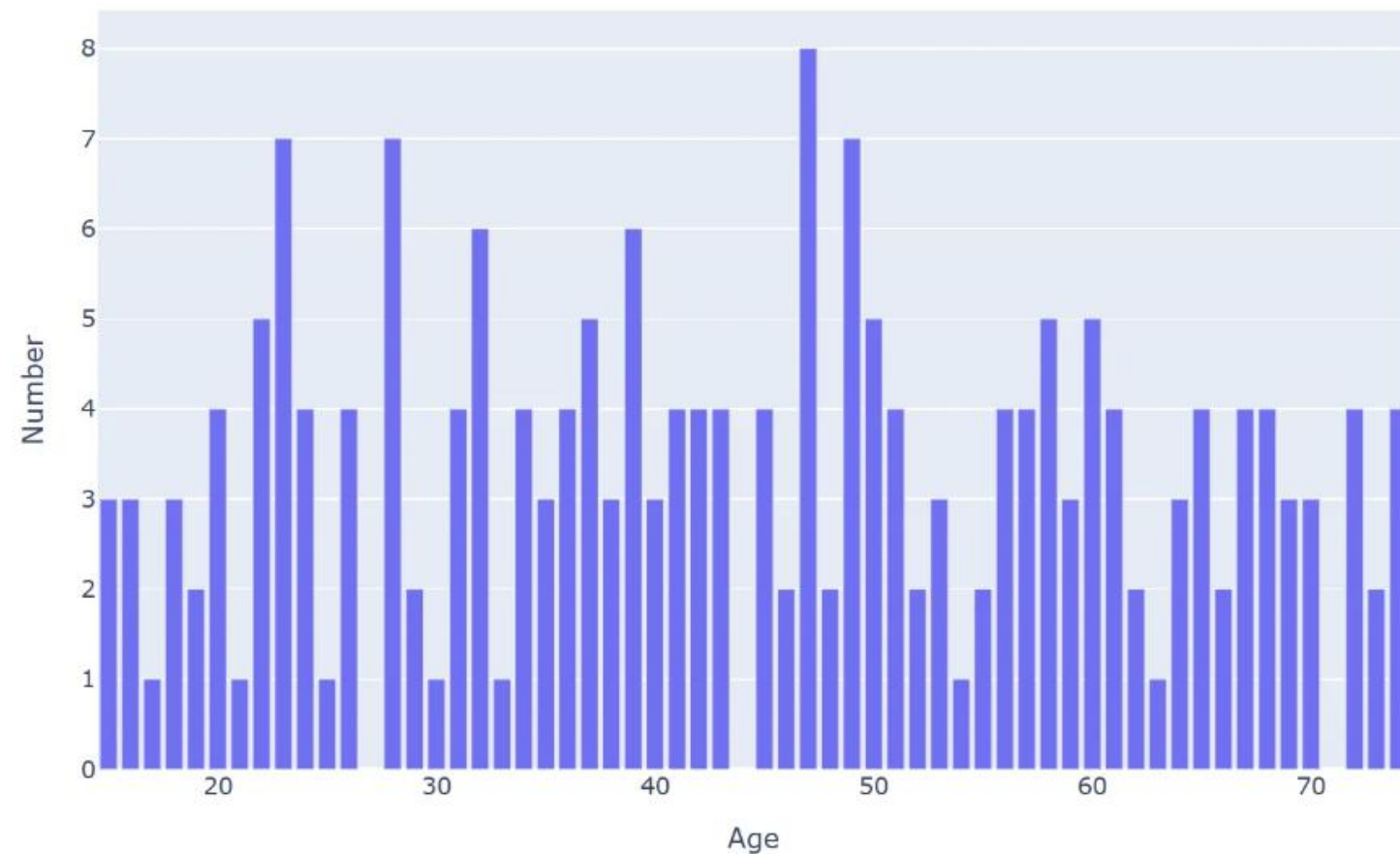
count	
Age	
47	8
23	7
28	7
49	7
32	6
39	6
50	5
60	5
22	5
37	5
58	5
43	4
41	4
57	4
65	4
61	4
74	4
45	4
56	4

```
# Dataset columns names
data.columns
```

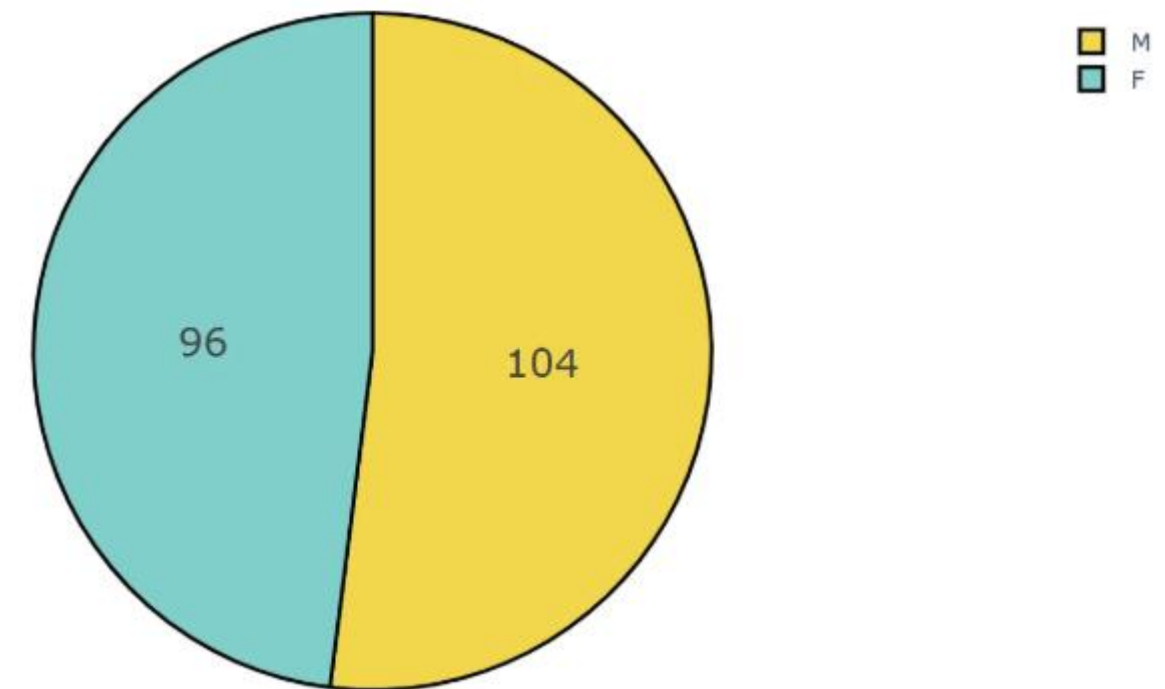

#02 Decision Tree And Random Forest Classifier Models

-2. 데이터 시각화

```
# I visualized the number of age in the dataset.  
dataAge = data["Age"].value_counts(dropna = False)  
npar_dataAge = np.array(dataAge)  
x = list(npar_dataAge)  
y = dataAge.value_counts().index  
  
DataAge = [{"Age": y, "Number": x}]  
DataAge = pd.DataFrame(DataAge)  
  
fig = px.bar(DataAge, x = "Age", y = "Number")  
fig.show()
```



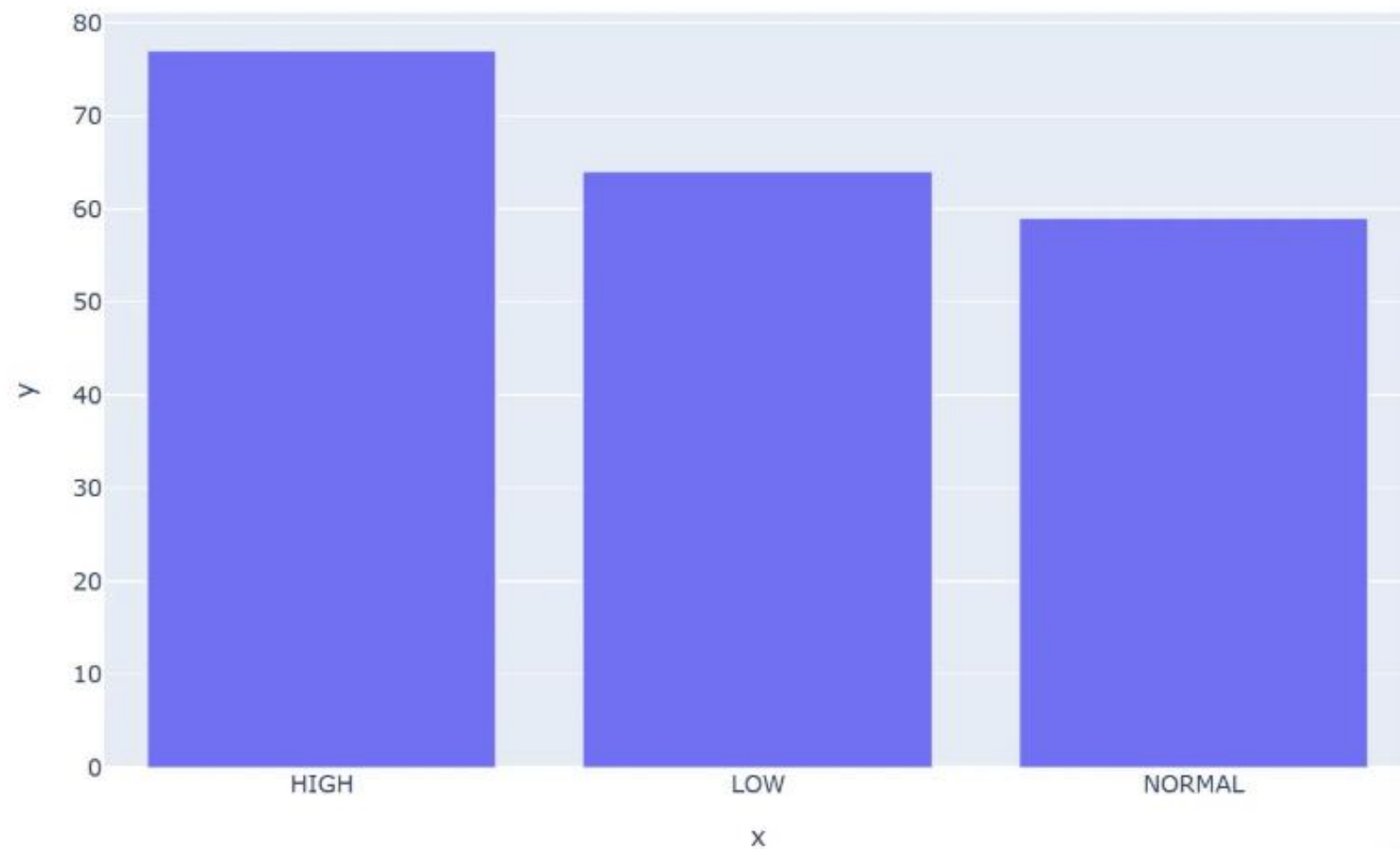
```
# I visualized the number of Sex in the dataset.  
colors = ['gold', 'mediumturquoise']  
  
fig = go.Figure(data = [go.Pie(labels= ['M', 'F'], values=[104, 96])])  
  
fig.update_traces(hoverinfo = 'label + percent', textinfo = 'value', textfont_size = 20,  
                  marker = dict(colors = colors, line = dict( color = '#000000', width = 2)))  
  
fig.show()
```



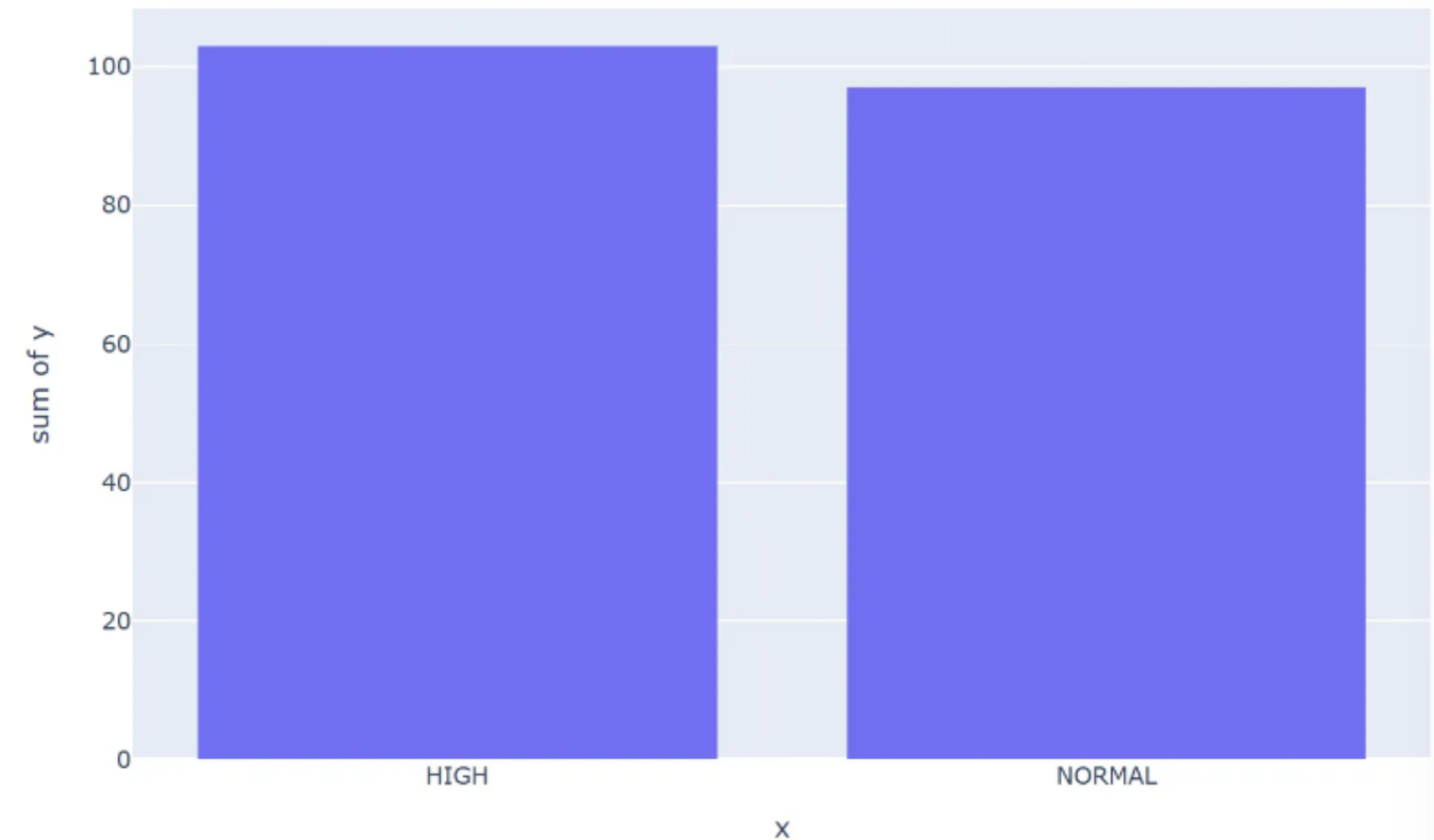
#02 Decision Tree And Random Forest Classifier Models

-2. 데이터 시각화

```
# I visualized the number of BP(Blood Pressure Levels) in the dataset.  
fig = px.bar(x = ["HIGH", "LOW", "NORMAL"], y = [77, 64, 59])  
  
fig.show()
```



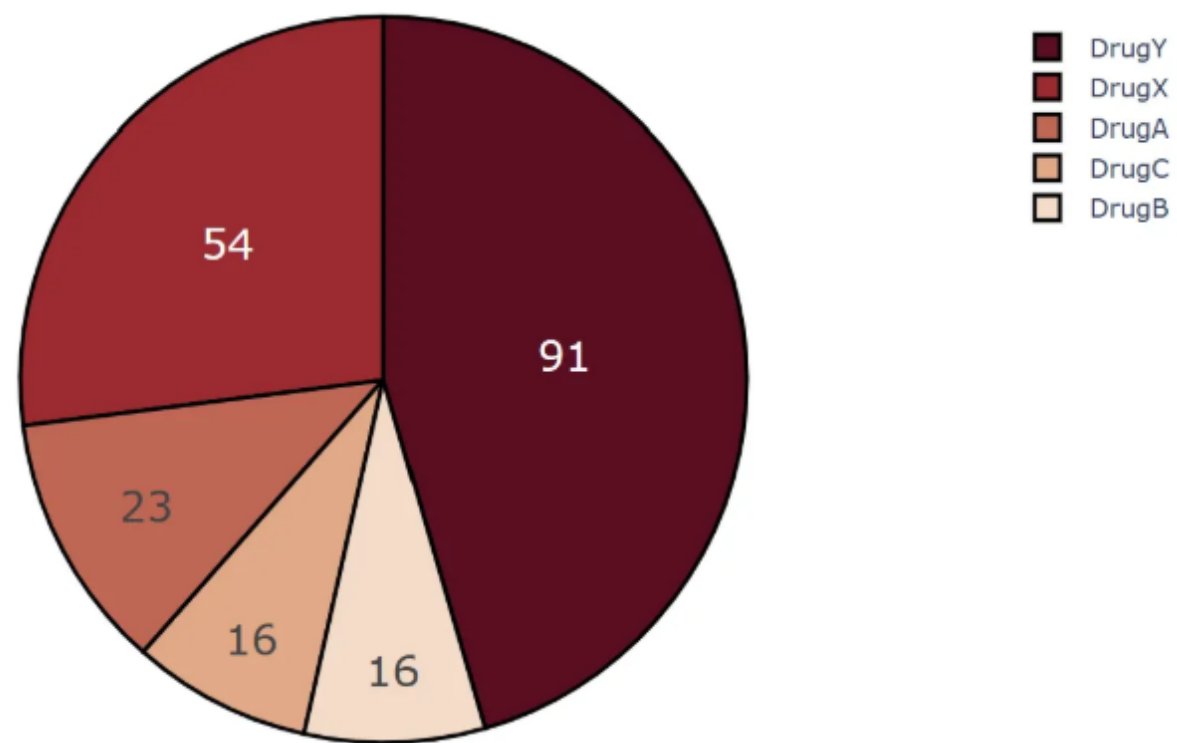
```
# I visualized the number of Cholesterol in the dataset.  
fig = px.histogram(x = ["HIGH", "NORMAL"], y = [103, 97])  
fig.show()
```



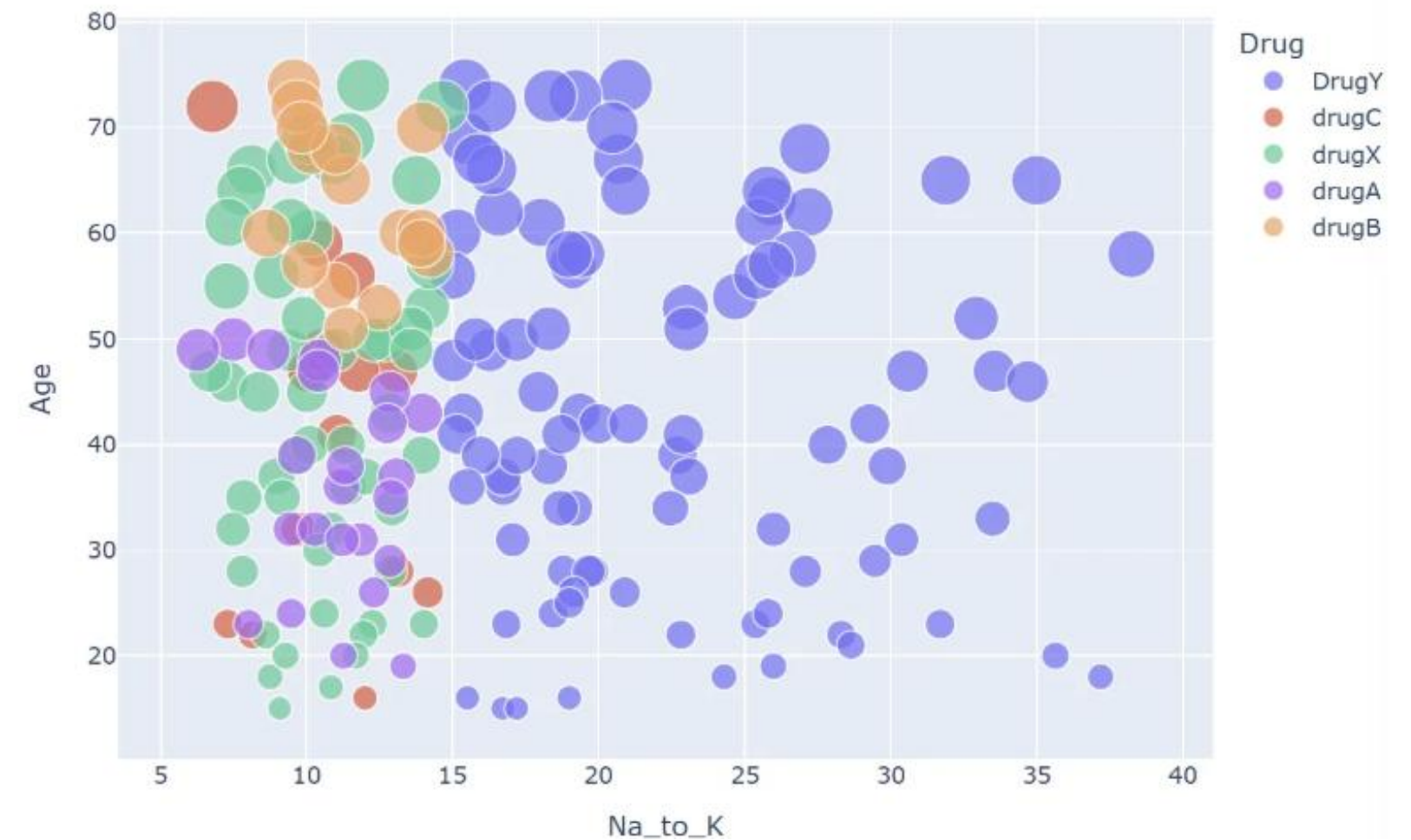
#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
# I visualized the number of Drug in the dataset.
fig = go.Figure(data = [go.Pie(labels=["DrugY", "DrugX", "DrugA", "DrugC", "DrugB"], values=[91, 54, 23, 16, 16],
fig.update_traces(hoverinfo = 'label + percent', textinfo = 'value', textfont_size = 20,
marker = dict(colors = px.colors.sequential.RdBu, line = dict( color = '#000000', width
fig.show()
```



```
# I visualized the relationship between age and Na_To_K(Na to Potassium Ration).
fig = px.scatter(data, x = "Na_to_K", y="Age", color="Drug",
size='Age', hover_data=['Na_to_K'])
fig.show()
```



#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
dataclass = pd.read_csv("drug200.csv")
```

dataclass						
	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	F	LOW	HIGH	11.567	drugC
196	16	M	LOW	HIGH	12.006	drugC
197	52	M	NORMAL	HIGH	9.894	drugX
198	23	M	NORMAL	NORMAL	14.020	drugX
199	40	F	LOW	NORMAL	11.349	drugX

200 rows × 6 columns

```
# I don't want to object type.
dataclass.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Age              200 non-null    int64
1   Sex              200 non-null    object
2   BP               200 non-null    object
3   Cholesterol       200 non-null    object
4   Na_to_K          200 non-null    float64
5   Drug             200 non-null    object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```


#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
# Change Age type.
# F = 1
# M = 0
dataclass.Sex = [1 if i == "F" else 0 for i in dataclass.Sex]

# Change BP(Blood Pressure Levels) type.
# LOW = 2
# NORMAL = 1
# HIGH = 0

import warnings
warnings.filterwarnings('ignore')

for i in range(0, len(dataclass.BP)):
    if dataclass.BP[i] == "LOW":
        dataclass.BP[i] = 2

    elif dataclass.BP[i] == "NORMAL":
        dataclass.BP[i] = 1

    else:
        dataclass.BP[i] = 0
```

dataclass						
	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	0	HIGH	25.355	DrugY
1	47	0	2	HIGH	13.093	drugC
2	47	0	2	HIGH	10.114	drugC
3	28	1	1	HIGH	7.798	drugX
4	61	1	2	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	1	2	HIGH	11.567	drugC
196	16	0	2	HIGH	12.006	drugC
197	52	0	1	HIGH	9.894	drugX
198	23	0	1	NORMAL	14.020	drugX
199	40	1	2	NORMAL	11.349	drugX

200 rows × 6 columns

#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
# Change Cholesterol type.
# HIGH = 1
# NORMAL = 0
dataclass.Cholesterol = [1 if i == "HIGH" else 0 for i in dataclass.Cholesterol]

# Change Drug type.
# DrugY = 4
# DrugX = 3
# DrugA = 2
# DrugC = 1
# DrugB = 0

import warnings
warnings.filterwarnings('ignore')

for i in range(0, len(dataclass)):
    if dataclass.Drug[i] == "DrugY":
        dataclass.Drug[i] = 4
    elif dataclass.Drug[i] == "drugX":
        dataclass.Drug[i] = 3
    elif dataclass.Drug[i] == "drugA":
        dataclass.Drug[i] = 2
    elif dataclass.Drug[i] == "drugC":
        dataclass.Drug[i] = 1
    else:
        dataclass.Drug[i] = 0
```

```
# It is the final form of the dataset.
dataclass
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	0	1	25.355	4
1	47	0	2	1	13.093	1
2	47	0	2	1	10.114	1
3	28	1	1	1	7.798	3
4	61	1	2	1	18.043	4
...	...	...	...	...	...	...
195	56	1	2	1	11.567	1
196	16	0	2	1	12.006	1
197	52	0	1	1	9.894	3
198	23	0	1	0	14.020	3
199	40	1	2	0	11.349	3

200 rows × 6 columns

#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
# I check again because possible unchanged types.  
dataclass.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 200 entries, 0 to 199  
Data columns (total 6 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   Age             200 non-null   int64  
1   Sex             200 non-null   int64  
2   BP              200 non-null   object  
3   Cholesterol      200 non-null   int64  
4   Na_to_K         200 non-null   float64  
5   Drug            200 non-null   object  
dtypes: float64(1), int64(3), object(2)  
memory usage: 9.5+ KB
```

```
# I saw don't change two columns of type and I change them to integer types.  
data_types_dict = {'BP': int, "Drug": int}
```

```
dataclass = dataclass.astype(data_types_dict)
```

```
dataclass.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 200 entries, 0 to 199  
Data columns (total 6 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   Age             200 non-null   int64  
1   Sex             200 non-null   int64  
2   BP              200 non-null   int64  
3   Cholesterol      200 non-null   int64  
4   Na_to_K         200 non-null   float64  
5   Drug            200 non-null   int64  
dtypes: float64(1), int64(5)  
memory usage: 9.5 KB
```

#02 Decision Tree And Random Forest Classifier Models

-3. 데이터 전처리

```
# x_data
x_data = dataclass.drop(["Drug"], axis = 1)

#y_data
y_data = dataclass.Drug.values
```

x_data					
	Age	Sex	BP	Cholesterol	Na_to_K
0	23	1	0	1	25.355
1	47	0	2	1	13.093
2	47	0	2	1	10.114
3	28	1	1	1	7.798
4	61	1	2	1	18.043
...	...	...	...	...	...
195	56	1	2	1	11.567
196	16	0	2	1	12.006
197	52	0	1	1	9.894
198	23	0	1	0	14.020
199	40	1	2	0	11.349

200 rows x 5 columns

```
y_data
array([4, 1, 1, 3, 4, 3, 4, 1, 4, 4, 1, 4, 4, 4, 3, 4, 3, 2, 1, 4, 4, 4,
       4, 4, 4, 4, 4, 3, 4, 4, 3, 0, 3, 4, 3, 3, 2, 3, 3, 3, 4, 0, 4, 3,
       3, 3, 2, 1, 4, 4, 4, 3, 4, 4, 0, 1, 0, 4, 3, 4, 4, 2, 4, 3, 0, 4,
       2, 3, 4, 4, 0, 4, 3, 4, 4, 4, 2, 4, 2, 3, 0, 3, 1, 2, 1, 0, 3, 4,
       4, 4, 4, 4, 4, 4, 4, 3, 4, 4, 4, 4, 2, 2, 1, 3, 4, 3, 3, 4, 0, 4,
       2, 3, 3, 3, 3, 4, 3, 3, 2, 4, 4, 4, 4, 4, 0, 4, 4, 3, 4, 3, 4, 4,
       3, 4, 4, 3, 0, 2, 0, 3, 2, 4, 0, 4, 2, 3, 3, 2, 3, 1, 2, 0, 3, 3,
       4, 1, 2, 4, 1, 3, 3, 0, 3, 4, 4, 4, 4, 3, 4, 2, 3, 3, 4, 4, 2, 4,
       2, 4, 4, 4, 4, 3, 3, 4, 4, 4, 0, 2, 4, 4, 4, 2, 4, 1, 4, 1, 1, 3,
       3, 3])

# Train test split
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.3, random_state=1)
```


#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

dtc = DecisionTreeClassifier()

# Fit the model
dtc.fit(x_train, y_train)

# Predict the x_test
predict = dtc.predict(x_test)

print('The accuracy of the Decision Tree is', metrics.accuracy_score(predict, y_test))
```

#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_DecisionTreeClassifier with GINI

```
DTC_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=0)
```

```
# Fit the model
```

```
DTC_gini.fit(x_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3, random_state=0)
```

```
y_pred_gini = DTC_gini.predict(x_test)
```

```
from sklearn.metrics import accuracy_score
```

```
print('Model accuracy score with criterion gini index: {0:0.4f}'.format(accuracy_score(y_test, y_pred_gini)))
```

```
Model accuracy score with criterion gini index: 0.9000
```

```
y_pred_train_gini = DTC_gini.predict(x_train)
```

```
y_pred_train_gini
```

```
array([3, 3, 2, 4, 2, 3, 3, 3, 4, 2, 0, 3, 4, 3, 4, 3, 4, 3, 4, 4, 4, 4,
       3, 4, 2, 4, 3, 3, 2, 3, 4, 2, 0, 0, 3, 3, 3, 3, 3, 4, 4, 4, 4, 3,
       3, 0, 0, 2, 4, 3, 4, 3, 4, 4, 3, 3, 4, 4, 2, 4, 4, 4, 2, 3, 4, 4,
       4, 3, 4, 3, 3, 2, 3, 2, 4, 4, 4, 4, 3, 4, 0, 3, 3, 4, 0, 4, 0, 4,
       4, 0, 4, 4, 2, 3, 4, 3, 2, 4, 3, 4, 4, 3, 4, 4, 4, 4, 4, 4, 3, 4,
       4, 2, 3, 3, 3, 4, 3, 4, 4, 4, 0, 4, 2, 3, 3, 2, 4, 4, 4, 4, 4, 3,
       2, 4, 3, 4, 2, 3, 2, 3])
```

#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_DecisionTreeClassifier with GINI

```
print('Training-set accuracy score: [0:0.4f]'.format(accuracy_score(y_train, y_pred_train_gini)))
```

Training-set accuracy score: 0.9143

```
# Print the scores on training and test set
```

```
print('Training set score: {:.4f}'.format(DTC_gini.score(x_train, y_train)))
```

```
print('Test set score: {:.4f}'.format(DTC_gini.score(x_test, y_test)))
```

#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_DecisionTreeClassifier with Entrophy

```
# instantiate the DecisionTreeClassifier model with criterion entropy
DTC_en = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=0)

# Fit the model
DTC_en.fit(x_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=0)
```

```
y_pred_en = DTC_en.predict(x_test)
```

```
print('Model accuracy score with criterion entropy: {0:0.4f}'.format(accuracy_score(y_test, y_pred_en))
```

```
Model accuracy score with criterion entropy: 0.9000
```


#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_DecisionTreeClassifier with Entrophy

```
y_pred_train_en = DTC_en.predict(x_train)
```

```
y_pred_train_en
```

```
array([3, 3, 2, 4, 2, 3, 3, 3, 4, 2, 0, 3, 4, 3, 4, 3, 4, 3, 4, 4, 4, 4,
       3, 4, 2, 4, 3, 3, 2, 3, 4, 2, 0, 0, 3, 3, 3, 3, 3, 4, 4, 4, 4, 3,
       3, 0, 0, 2, 4, 3, 4, 3, 4, 4, 3, 3, 4, 4, 2, 4, 4, 4, 2, 3, 4, 4,
       4, 3, 4, 3, 3, 2, 3, 2, 4, 4, 4, 4, 3, 4, 0, 3, 3, 4, 0, 4, 0, 4,
       4, 0, 4, 4, 2, 3, 4, 3, 2, 4, 3, 4, 4, 3, 4, 4, 4, 4, 4, 4, 3, 4,
       4, 2, 3, 3, 3, 4, 3, 4, 4, 4, 0, 4, 2, 3, 3, 2, 4, 4, 4, 4, 4, 3,
       2, 4, 3, 4, 2, 3, 2, 3])
```

```
print('Training-set accuracy score: {0:0.4f}'.format(accuracy_score(y_train, y_pred_train_en)))
```

```
Training-set accuracy score: 0.9143
```

```
# print the scores on training and test set
```

```
print('Training set score: {:.4f}'.format(DTC_en.score(x_train, y_train)))
```

```
print('Test set score: {:.4f}'.format(DTC_en.score(x_test, y_test)))
```

#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(random_state = 0)

# Fit the model
rfc.fit(x_train, y_train)

# Predict the model
predict = rfc.predict(x_test)

print('The accuracy of the Random Forest is', metrics.accuracy_score(predict, y_test))
```

```
from sklearn.ensemble import RandomForestClassifier

rfc_100 = RandomForestClassifier(n_estimators=100, random_state=0)

# Fit the model
rfc_100.fit(x_train, y_train)

# Predict the model
predict = rfc_100.predict(x_test)

print('The accuracy of the Random Forest is', metrics.accuracy_score(predict, y_test))
```

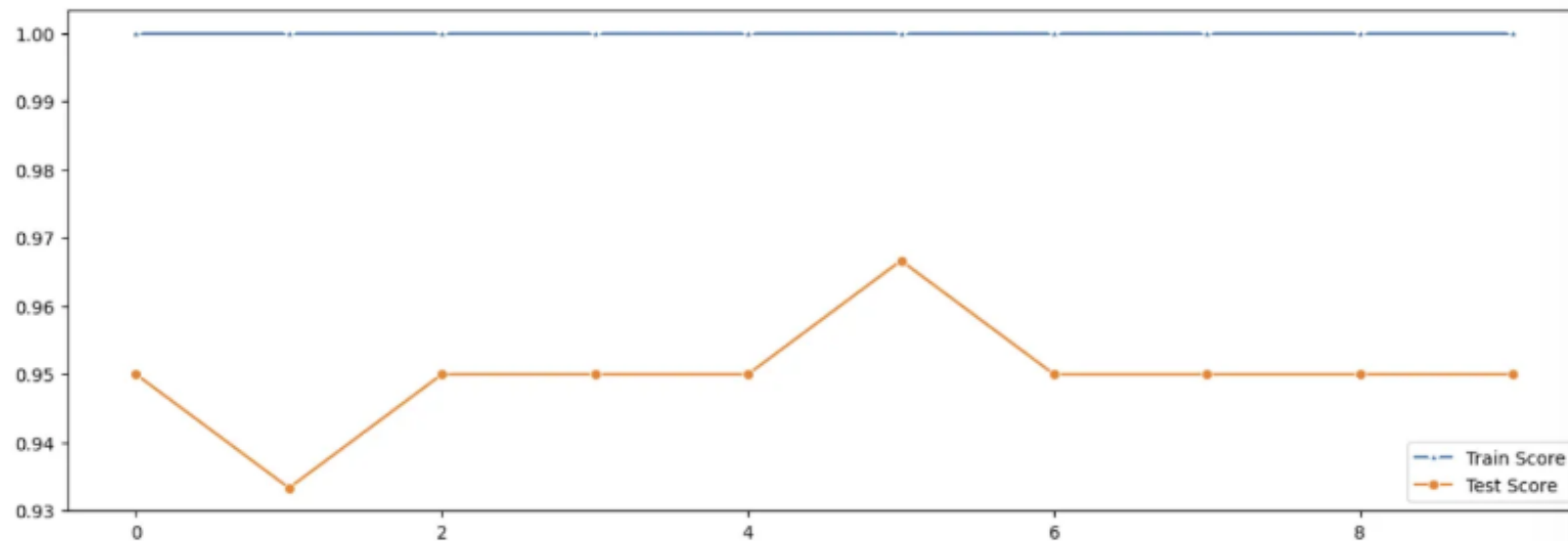
#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_RandomForestClassifier with n_estimators

```
# In this chapter, I want to find the best random_state value.
test_score_list = []
train_score_list = []

for i in range(0,10):
    rfc2 = RandomForestClassifier(random_state=i)
    rfc2.fit(x_train, y_train)
    test_score_list.append(rfc2.score(x_test, y_test))
    train_score_list.append(rfc2.score(x_train, y_train))

plt.figure(figsize=(15,5))
p = sns.lineplot(x=range(0, 10), y=train_score_list, marker='*', label='Train Score')
p = sns.lineplot(x=range(0, 10), y=test_score_list, marker='o', label='Test Score')
```

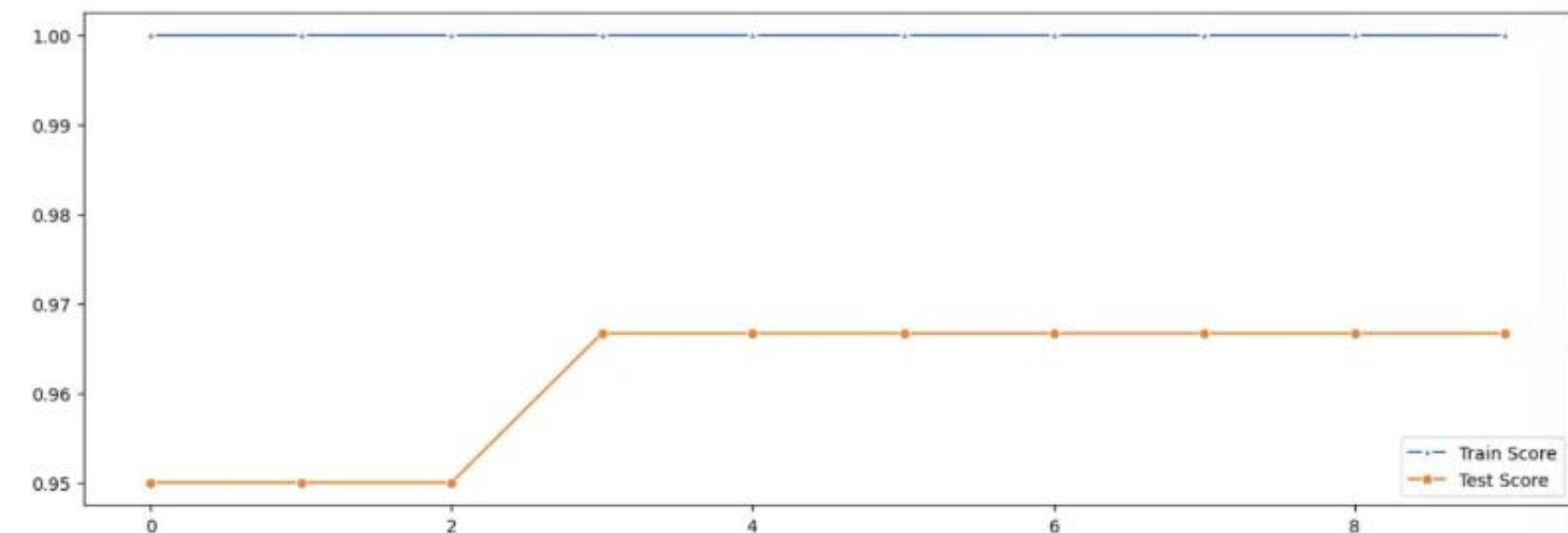


```
# I found the best random_state value is five. Now I will find the best n_estimators parameter.
test_score_list = []
train_score_list = []

list_n_estimators = [10,20,30,40,50,60,70,80,90,100]

for i in range(0,len(list_n_estimators)):
    rfc3 = RandomForestClassifier(n_estimators=list_n_estimators[i], random_state=5)
    rfc3.fit(x_train, y_train)
    test_score_list.append(rfc3.score(x_test, y_test))
    train_score_list.append(rfc3.score(x_train, y_train))

plt.figure(figsize=(15,5))
p=sns.lineplot(x=range(0,len(list_n_estimators)),y=train_score_list,marker='*',label='Train Score')
p=sns.lineplot(x=range(0,len(list_n_estimators)),y=test_score_list,marker='o',label='Test Score')
```



#02 Decision Tree And Random Forest Classifier Models

-4. 모델링_RandomForestClassifier with the best parameters

```
last_rfc = RandomForestClassifier(n_estimators=100, random_state=5)

# fit the model
last_rfc.fit(x_train,y_train)

predict = last_rfc.predict(x_test)

print('The accuracy of the Random Forest is',metrics.accuracy_score(predict,y_test))
```

The accuracy of the Random Forest is 0.9666666666666667

```
y_pred_en = last_rfc.predict(x_test)
```

```
print('Model accuracy score with best parameters: {0:0.4f}'.format(accuracy_score(y_test, y_pred_en)))
```

Model accuracy score with best parameters: 0.9667

```
y_pred_train_en = last_rfc.predict(x_train)
```

```
y_pred_train_en
```

```
array([3, 3, 2, 4, 2, 3, 3, 3, 4, 2, 0, 1, 4, 1, 4, 3, 4, 3, 4, 4, 4, 4,
       3, 4, 2, 4, 1, 3, 2, 3, 4, 2, 0, 0, 3, 3, 1, 3, 3, 4, 4, 4, 4, 3,
       1, 0, 0, 2, 4, 3, 4, 1, 4, 4, 1, 3, 4, 4, 2, 4, 4, 4, 2, 3, 4, 4,
       4, 3, 4, 1, 3, 2, 3, 2, 4, 4, 4, 4, 3, 4, 0, 3, 3, 4, 0, 4, 0, 4,
       4, 0, 4, 4, 2, 3, 4, 3, 2, 4, 1, 4, 4, 3, 4, 4, 4, 4, 4, 1, 4,
       4, 2, 3, 1, 1, 4, 3, 4, 4, 4, 0, 4, 2, 3, 3, 2, 4, 4, 4, 4, 4, 3,
       2, 4, 3, 4, 2, 3, 2, 3])
```

```
print('Training-set accuracy score: {0:0.4f}'.format(accuracy_score(y_train, y_pred_train_en)))
```

Training-set accuracy score: 1.0000

```
# print the scores with best parameters on training and test set
```

```
print('Training set score: {:.4f}'.format(last_rfc.score(x_train, y_train)))
```

```
print('Test set score: {:.4f}'.format(last_rfc.score(x_test, y_test)))
```


#02 Decision Tree And Random Forest Classifier Models

-5. 평가_Confusion Matrix

```
# For Decision Tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

cm_des = DecisionTreeClassifier()

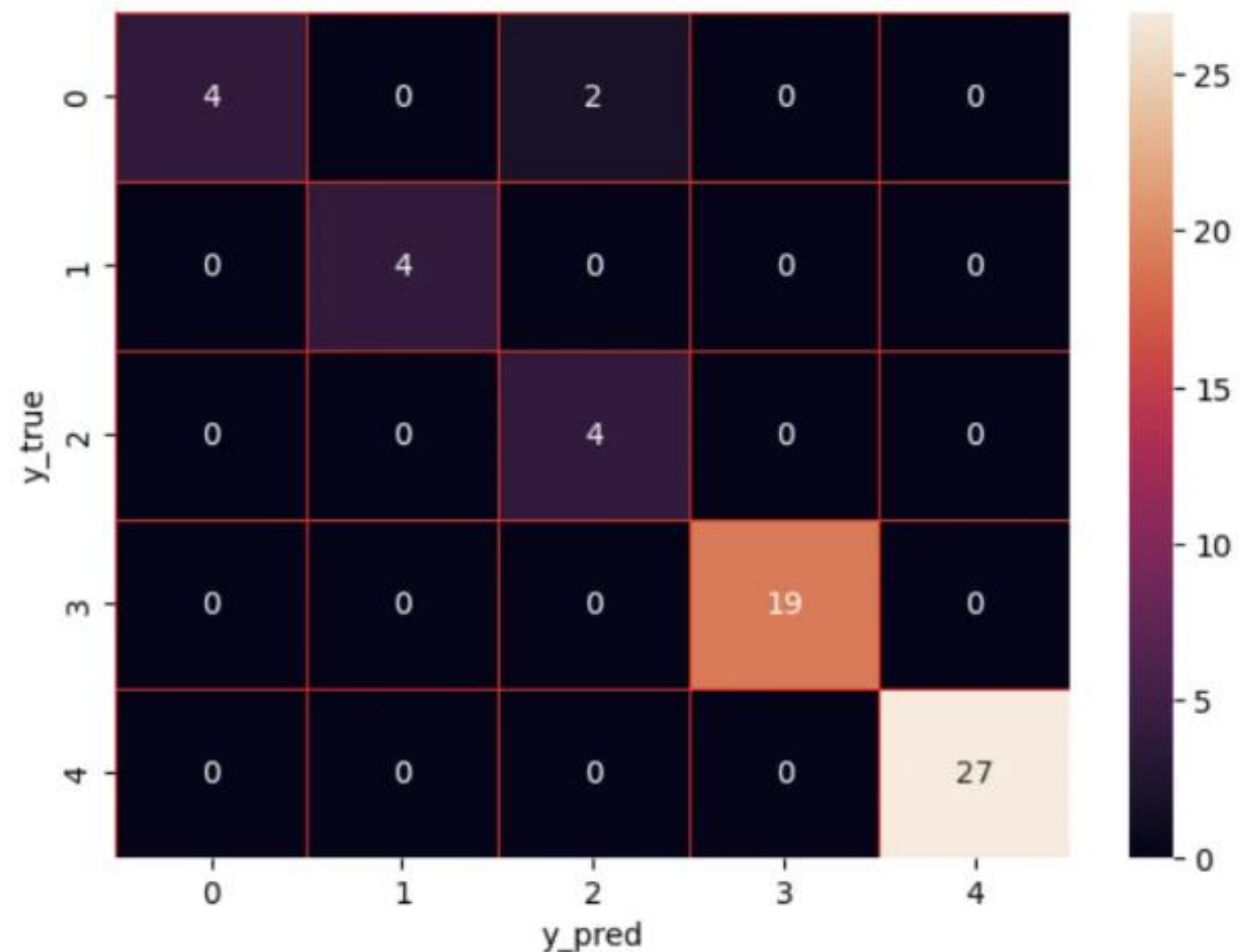
# fit the model
cm_des.fit(x_train,y_train)

y_pred_cm = cm_des.predict(x_test)
y_true = y_test

cm_des1 = confusion_matrix( y_true, y_pred_cm)
cm_des1

array([[ 4,  0,  2,  0,  0],
       [ 0,  4,  0,  0,  0],
       [ 0,  0,  4,  0,  0],
       [ 0,  0,  0, 19,  0],
       [ 0,  0,  0,  0, 27]])
```

```
f, ax = plt.subplots(figsize = (7,5))
sns.heatmap(cm_des1, annot = True, linewidths=0.5, linecolor="red", fmt = ".0f", ax = ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()
```



#02 Decision Tree And Random Forest Classifier Models

-5. 평가_Confusion Matrix

```
cm_des_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=0)

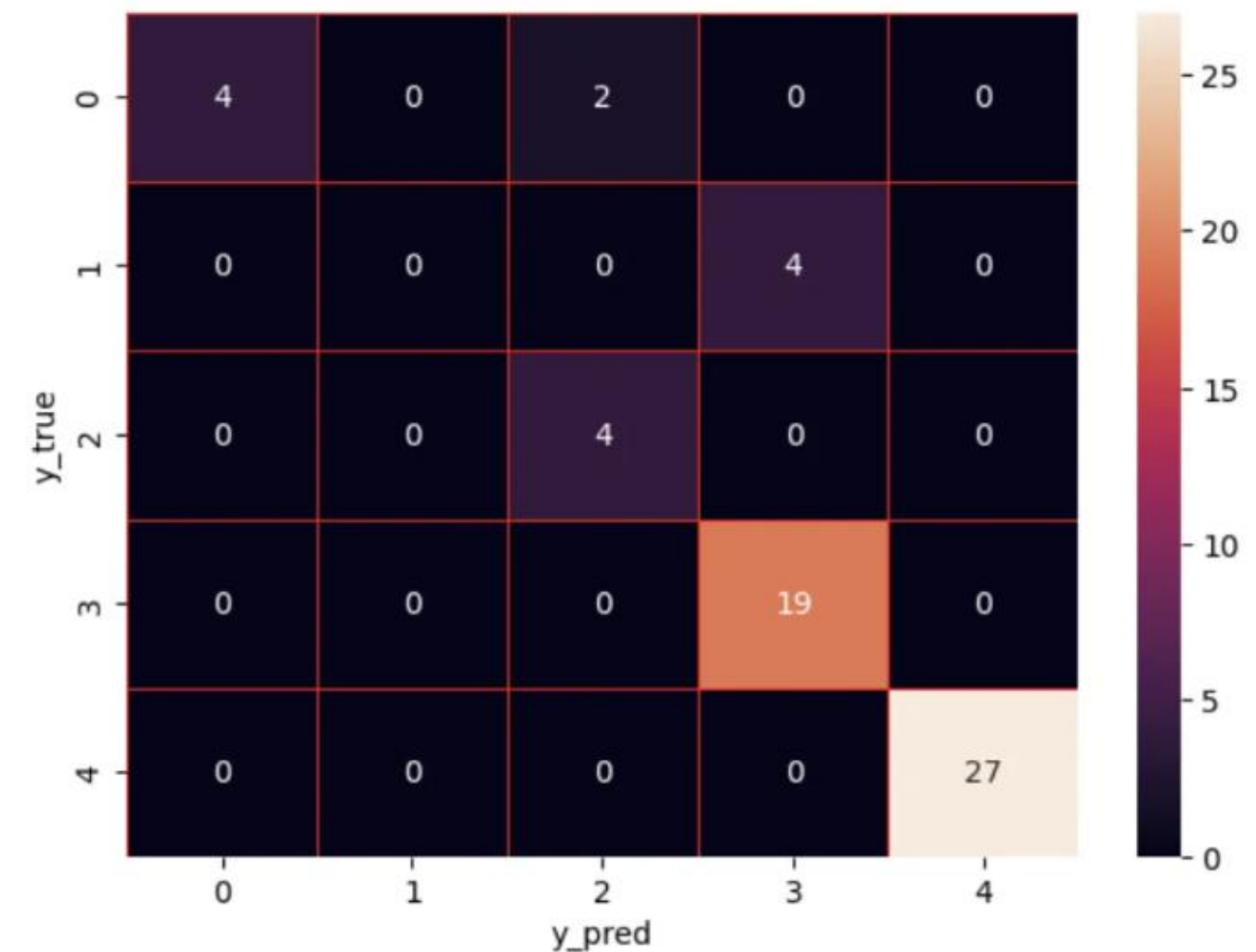
# fit the model
cm_des_gini.fit(x_train,y_train)

y_pred_cm = cm_des_gini.predict(x_test)
y_true = y_test

cm_des2 = confusion_matrix( y_true, y_pred_cm)
cm_des2

array([[ 4,  0,  2,  0,  0],
       [ 0,  0,  0,  4,  0],
       [ 0,  0,  4,  0,  0],
       [ 0,  0,  0, 19,  0],
       [ 0,  0,  0,  0, 27]])
```

```
f, ax = plt.subplots(figsize = (7,5))
sns.heatmap(cm_des2, annot = True, linewidths=0.5, linecolor="red", fmt = ".0f", ax = ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()
```



#02 Decision Tree And Random Forest Classifier Models

-5. 평가_Confusion Matrix

```
cm_last_rfc = RandomForestClassifier(n_estimators=100, random_state=5)

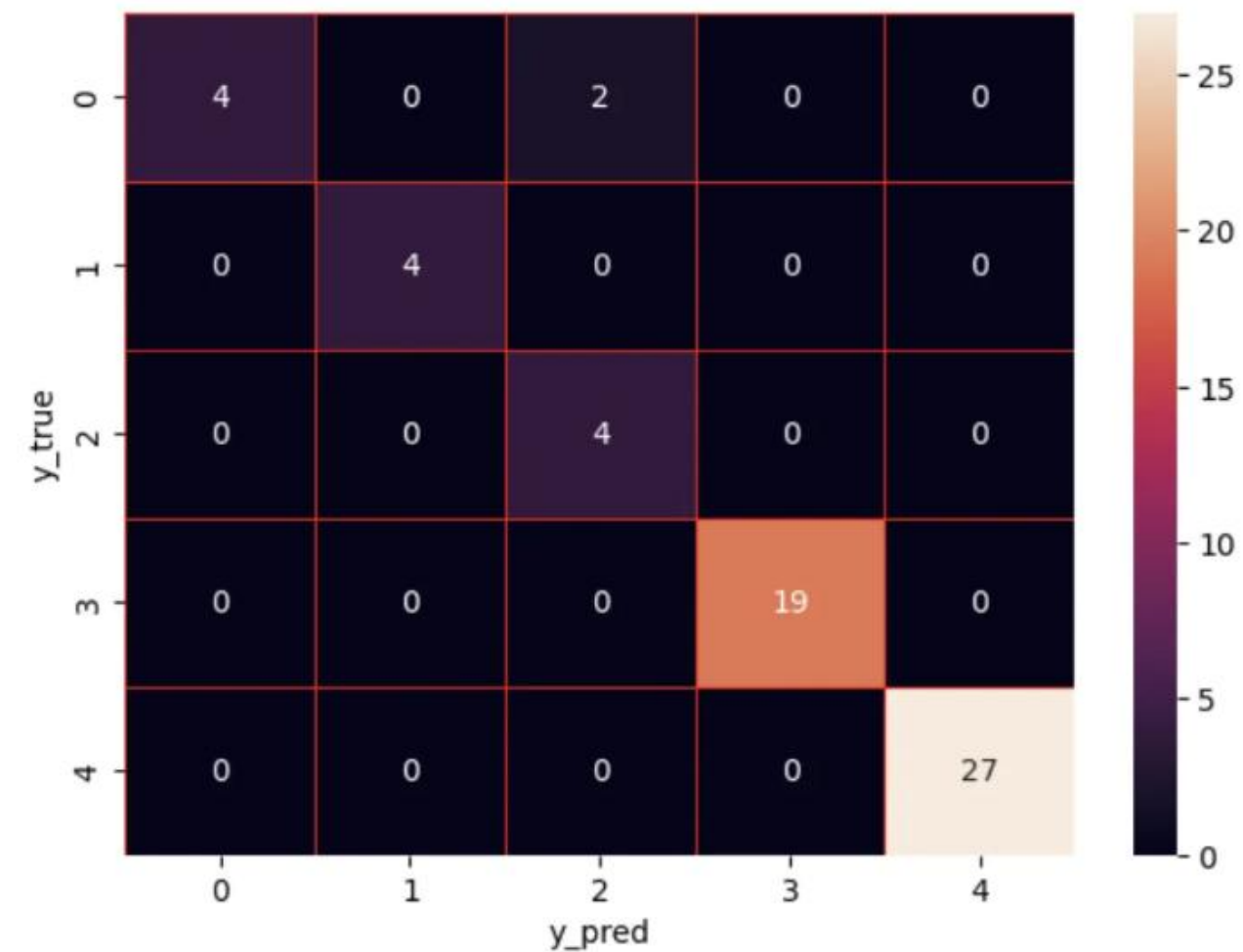
# Fit The Model
cm_last_rfc.fit(x_train, y_train)

y_pred_cm = cm_last_rfc.predict(x_test)
y_true = y_test

cm_rfc = confusion_matrix(y_true, y_pred_cm)
cm_rfc

array([[ 4,  0,  2,  0,  0],
       [ 0,  4,  0,  0,  0],
       [ 0,  0,  4,  0,  0],
       [ 0,  0,  0, 19,  0],
       [ 0,  0,  0,  0, 27]])
```

```
f, ax = plt.subplots(figsize = (7,5))
sns.heatmap(cm_rfc, annot = True, linewidths=0.5, linecolor="red", fmt = ".0f", ax = ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()
```



4.10 캐글 신용카드 사기 검출



Kaggle 데이터 세트

<https://www.kaggle.com/mlg-ulb/creditcardfraud>

신용카드 데이터 세트 -> 신용 카드 사기 검출 분류 실습

 MACHINE LEARNING GROUP - ULB AND 1 COLLABORATOR · UPDATED 7 YEARS AGO

 12027  Code  Download  

Credit Card Fraud Detection

Anonymized credit card transactions labeled as fraudulent or genuine



Data Card Code (5294) Discussion (108) Suggestions (0)

About Dataset

Context

It is important that credit card companies are able to recognize fraudulent credit card transactions so that customers are not charged for items that they did not purchase.

Usability ⓘ

8.53

License

Database: Open Database, Cont...

Expected update frequency

Kaggle 데이터 세트

Creditcard.csv 레이블 class 속성

0 : 사기가 아닌 정상적인 신용카드 트랜잭션 데이터

1: 신용카드 사기 트랜잭션 - 전체 데이터 중 약 0.172% 차지

⇒ 이상 검출 데이터 세트!

이상 검출 데이터 세트 (ex 사기 검출)

이상 현상은 전체 데이터에서의 비중이 매우 적을 수밖에 없음

-> 레이블 값이 극도로 불균형한 분포를 가지기 쉬움

언더 샘플링과 오버 샘플링의 이해 ☆

레이블이 불균형한 분포를 가진 데이터 세트

: 학습 시 예측 성능 문제 발생

정상 레이블 데이터의 수가 매우 많음 -> 일반적으로 정상 레이블에 치우친 학습을 수행하여 제대로 된 이상 데이터 검출이 어려워짐

=> 불균형한 레이블 값 분포를 해결하기 위한 방안 필요: Oversampling & Undersampling

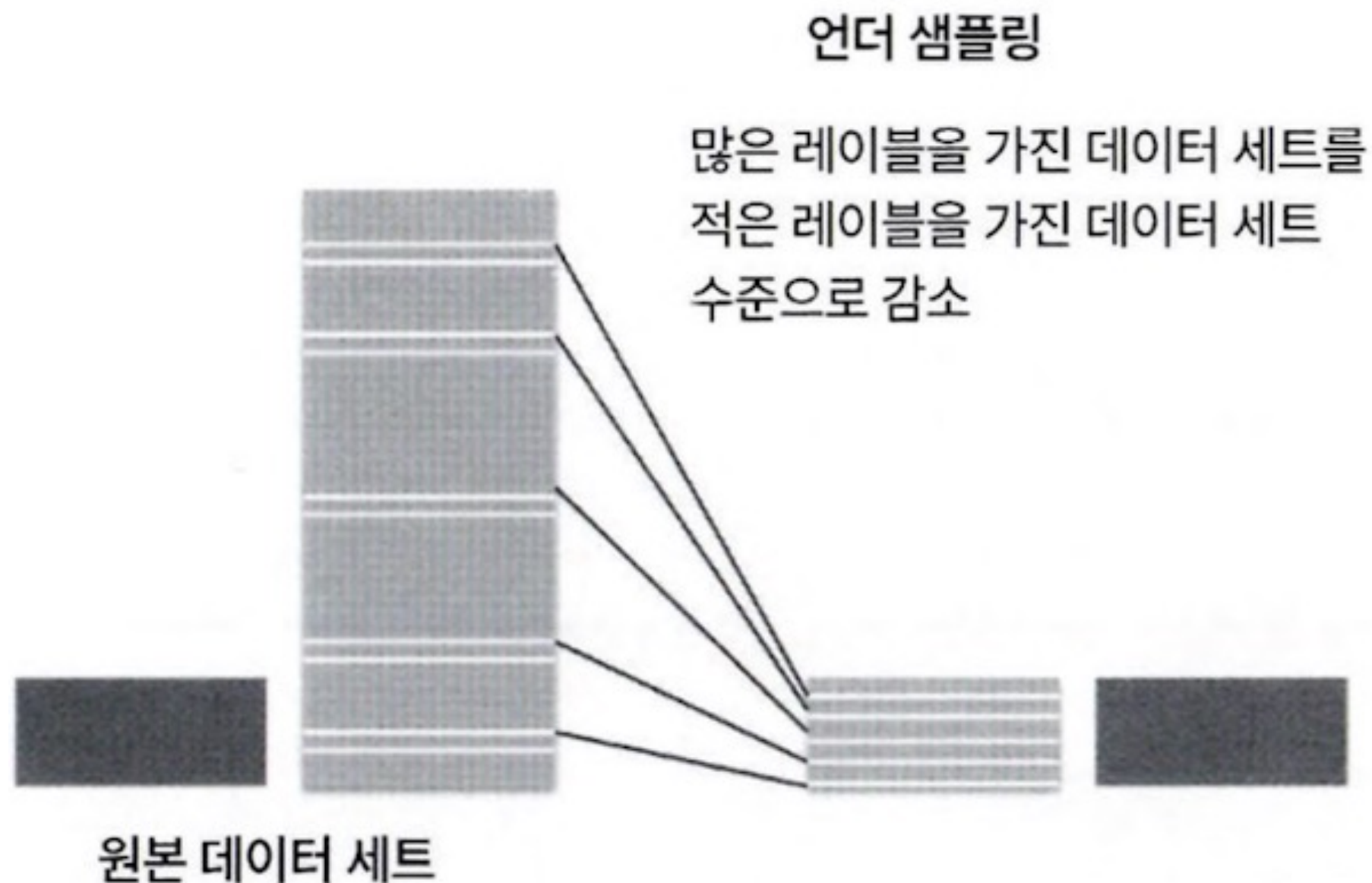
언더 샘플링과 오버 샘플링의 이해 ☆

언더 샘플링

: 많은 데이터 세트를 **적은 데이터 세트 수준으로 감소시키는 방식**

과도하게 정상 레이블로 학습하는 부작용 개선

But, 많은 정상 레이블 데이터를 감소시키므로 제대로 된 학습 X



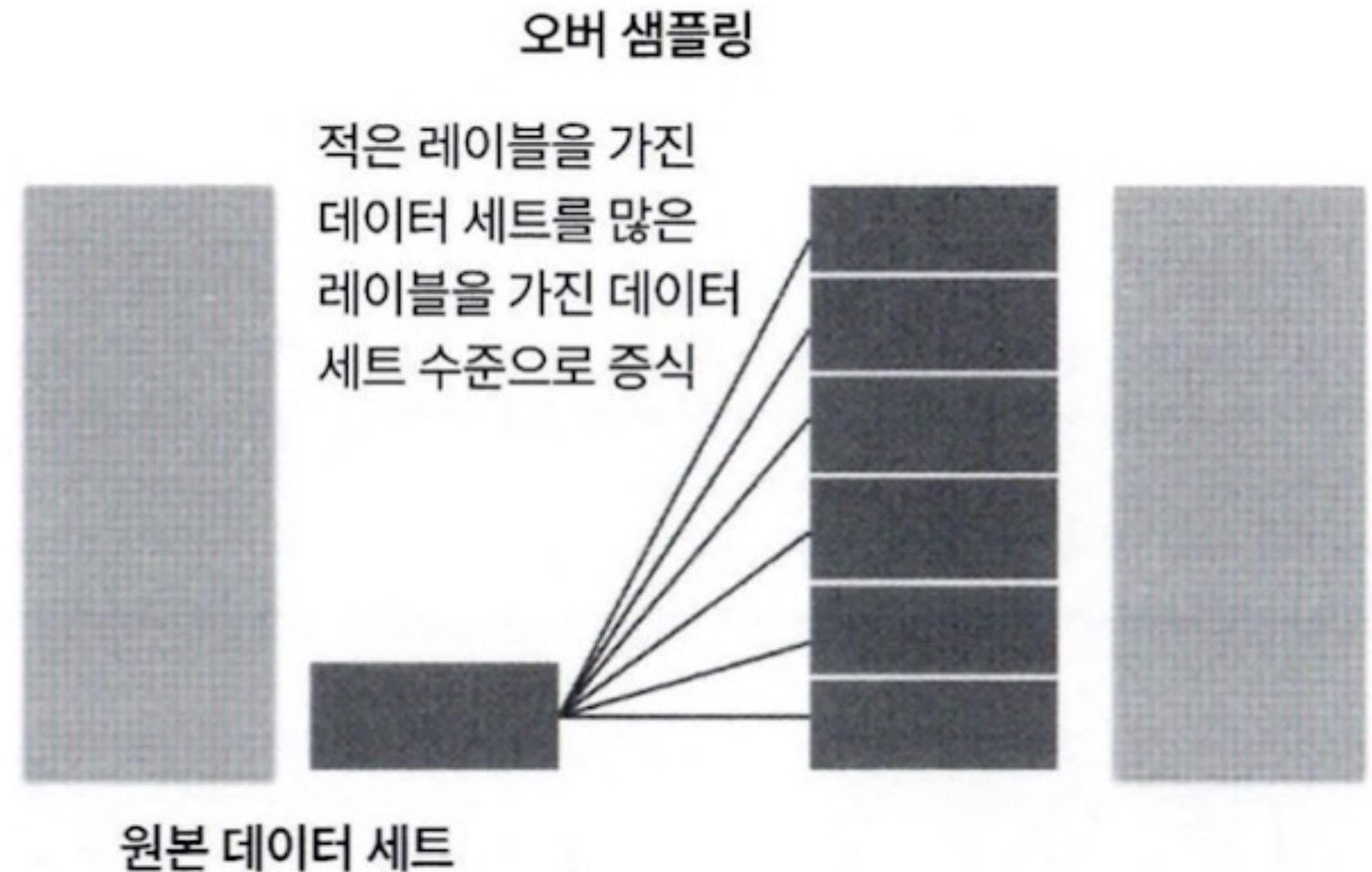
언더 샘플링과 오버 샘플링의 이해 ☆

오버 샘플링

: 이상 데이터와 같이 **적은 데이터 세트를 증식하여** 학습을 위한 충분한 데이터를 확보하는 방식

동일한 데이터를 단순히 증식할 경우 -> 과적합

원본 데이터의 피쳐 값을 아주 약간 변경하며 증식
- SMOTE 방식 사용



언더 샘플링과 오버 샘플링의 이해 ☆

SMOTE (Synthetic Minority Over-sampling Technique)

: 적은 데이터 세트에 있는 개별 데이터들의 **K 최근접 이웃** 을 찾아, 이 데이터와 **K개의 이웃들의 차이를 일정 값으로** 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터들을 생성하는 방식

- 정밀도보다 재현율을 높이고 싶을 때 사용하면 좋음

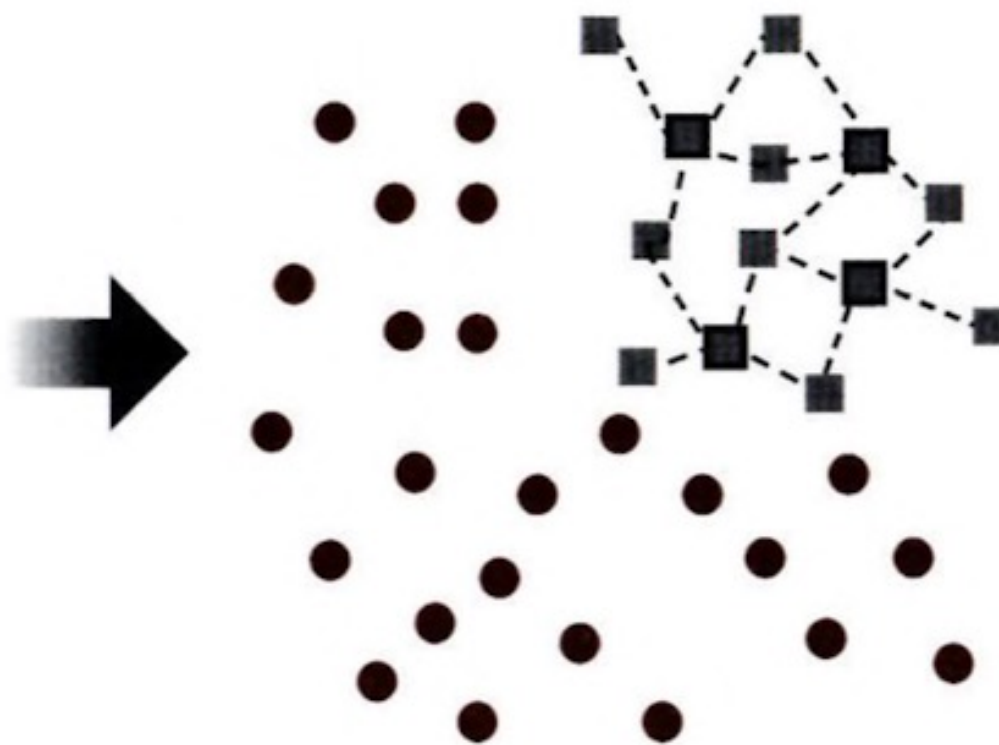
- 구현 파이썬 패키지: imbalanced-learn

```
[ ] !pip install imbalanced-learn
```

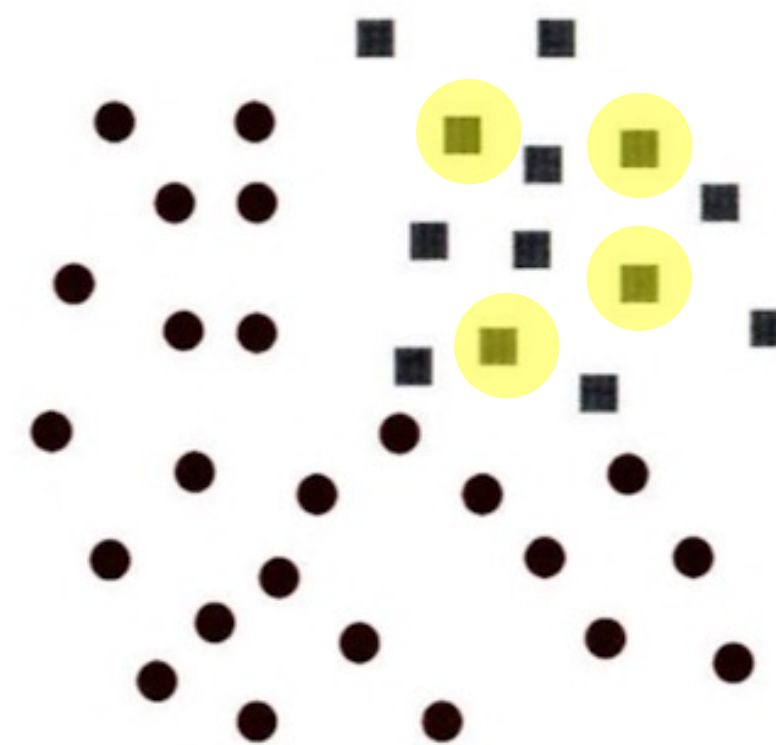
① 원본 데이터



② K 최근접 이웃으로 데이터 신규 증식 대상 설정



③ 신규 증식하여 오버 샘플링 완료



데이터 일차 가공 및 모델 학습/예측/평가

DataFrame 업로드

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline

card_df=pd.read_csv('/content/drive/MyDrive/EURON/test/creditcard.csv')
card_df.head(3)
```

데이터 일차 가공 및 모델 학습/예측/평가

Feature 의미 파악

Time: 데이터 생성 관련 작업용 속성 -> 제거

V1~V28: 의미를 알 수 없음

Amount: 신용카드 트랜잭션 금액

Class: 0 – 정상, 1– 사기 트랜잭션

	Time	V1	V2	V3	V4
0	0.0	-1.359807	-0.072781	2.536347	1.378155
1	0.0	1.191857	0.266151	0.166480	0.448154
2	1.0	-1.358354	-1.340163	1.773209	0.379780

3 rows x 31 columns

V26	V27	V28	Amount	Class
-0.189115	0.133558	-0.021053	149.62	0
0.125895	-0.008983	0.014724	2.69	0
-0.139097	-0.055353	-0.059752	378.66	0

데이터 일차 가공 및 모델 학습/예측/평가

get_preprocessed_df()

: df.drop('Time')

```
from sklearn.model_selection import train_test_split

# 인자로 입력받은 DataFrame을 복사한 뒤 Time 칼럼만 삭제하고 복사된 DataFrame 반환
def get_preprocessed_df(df=None):
    df_copy=df.copy()
    df_copy.drop( 'Time',axis=1,inplace=True)
    return df_copy
```

데이터 일차 가공 및 모델 학습/예측/평가

get_train_test_dataset()

: get_preprocessed_df() 호출 후, 학습 피처/레이블 데이터 세트, 테스트 피처/레이블 데이터 세트 반환

```
# 사전 데이터 가공 후 학습과 테스트 데이터 세트를 반환하는 함수
def get_train_test_dataset(df=None):
    # 인자로 입력된 DataFrame의 사전 데이터 가공이 완료된 복사 DataFrame 반환
    df_copy=get_preprocessed_df(df)
    # DataFrame의 맨 마지막 칼럼이 레이블, 나머지는 피처들
    X_features=df_copy.iloc[:, :-1]
    y_target=df_copy.iloc[:, -1]
    # train_test_split()으로 학습과 테스트 데이터 분할, stratify=y_target으로 Stratified 기반 분할
    X_train,X_test,y_train,y_test=train_test_split(X_features,y_target,test_size=0.3,random_state=0,stratify=y_target)
    # 학습과 테스트 데이터 세트 반환
    return X_train, X_test, y_train, y_test
```

```
X_train,X_test,y_train,y_test=get_train_test_dataset(card_df)
```

데이터 일차 가공 및 모델 학습/예측/평가

get_model_train_eval()

: 반복적으로 모델을 변경하여 학습/예측/평가를 위한 별도의 함수 생성

```
# 인자로 사이킷런의 Estimator 객체와 학습/테스트 데이터 세트를 입력받아서 학습/예측/평가 수행
def get_model_train_eval(model, ftr_train=None, ftr_test=None, tgt_train=None, tgt_test=None):
    model.fit(ftr_train, tgt_train)
    pred=model.predict(ftr_test)
    pred_proba=model.predict_proba(ftr_test)[:,1]
    get_clf_eval(tgt_test, pred, pred_proba)
```

데이터 일차 가공 및 모델 학습/예측/평가

Logistic Regression vs LightGBM 재현율, ROC-AUC 비교

- Logistic Regression

오차 행렬

```
[[85280    15]
 [    56    92]]
```

정확도: 0.9992, 정밀도: 0.8598, 재현율: 0.6216, F1: 0.7216, AUC: 0.9738

- LightGBM

오차 행렬

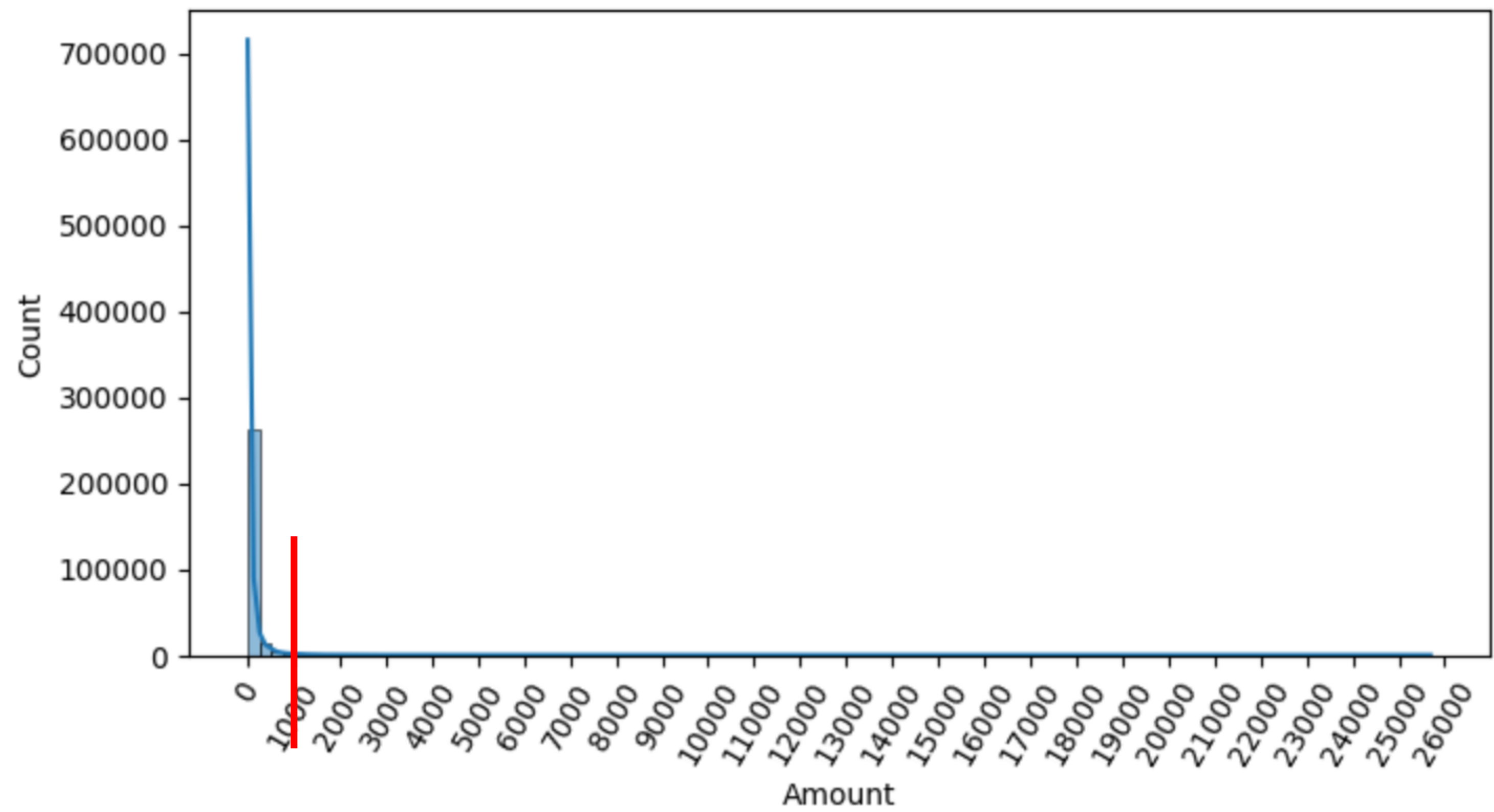
```
[[85290     5]
 [    36   112]]
```

정확도: 0.9995, 정밀도: 0.9573, 재현율: 0.7568, F1: 0.8453, AUC: 0.9790

=> LightGBM의 수치가 더 높음

데이터 분포도 변환 후 학습/예측/평가

Amount 피쳐 분포도



데이터 분포도 변환 후 학습/예측/평가

Amount를 표준 정규 분포 형태로 변환

```
from sklearn.preprocessing import StandardScaler
# 사이킷런의 StandardScaler를 이용해 정규 분포 형태로 Amount 피쳐값 변환하는 로직으로 수정
def get_preprocessed_df(df=None):
    df_copy=df.copy()
    scaler=StandardScaler()
    amount_n=scaler.fit_transform(df_copy['Amount'].values.reshape(-1,1))
    # 변환된 Amount를 Amount_scaled로 피쳐명 변경후 DataFrame맨 앞 칼럼으로 입력
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    # 기존 Time, Amount 피쳐 삭제
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

데이터 분포도 변환 후 학습/예측/평가

로지스틱 회귀 & LightGBM 학습 성능 비교

- 표준 정규 분포 변환

로지스틱 회귀 예측 성능

오차 행렬

```
[[85281      14]
 [    55      93]]
```

정확도: 0.9992, 정밀도: 0.8692, 재현율: 0.6284, F1: 0.7294, AUC: 0.9706

LightGBM 예측 성능

오차 행렬

```
[[85290        5]
 [    37     111]]
```

정확도: 0.9995, 정밀도: 0.9569, 재현율: 0.7500, F1: 0.8409, AUC: 0.9779

로지스틱 회귀: 정밀도 & 재현율 저하

LightGBM: 정밀도 & 재현율 저하 but 큰 성능 변화 X

데이터 분포도 변환 후 학습/예측/평가

Amount를 로그변환

```
def get_preprocessed_df(df=None):  
    df_copy=df.copy()  
    # 넘파이의 log1p()를 이용해 amount를 로그 변환  
    amount_n=np.log1p(df_copy['Amount'])  
    df_copy.insert(0, 'Amount_Scaled', amount_n)  
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)  
    return df_copy
```


데이터 분포도 변환 후 학습/예측/평가

로지스틱 회귀 & LightGBM 학습 성능 비교

- 로그 변환

로지스틱 회귀 예측 성능

오차 행렬

```
[[85282    13]
 [   59    89]]
```

정확도: 0.9992, 정밀도:0.8725, 재현율: 0.6014, F1: 0.7120, AUC:0.9734

LightGBM 예측 성능

오차 행렬

```
[[85290     5]
 [   35   113]]
```

정확도: 0.9995, 정밀도:0.9576, 재현율: 0.7635, F1: 0.8496, AUC:0.9796

로지스틱 회귀: 정밀도 향상 & 재현율 저하

LightGBM: 재현율 향상

=> 레이블이 불균형한 데이터 세트: 로지스틱 회귀는 데이터 변환시 불안정한 성능 결과를 보여줌

이상치 데이터 제거 후 모델 학습/예측/평가

이상치 데이터 (Outlier)

: 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터

- 이상치로 인해 성능에 영향을 받음
- 레이블과 **상관성이 높은 피처**를 위주로 이상치를 검출하는 것이 좋음
- IQR 방식으로 제거

IQR(Inter Quantile Range)

: **사분위 값의 편차**를 이용하는 기법

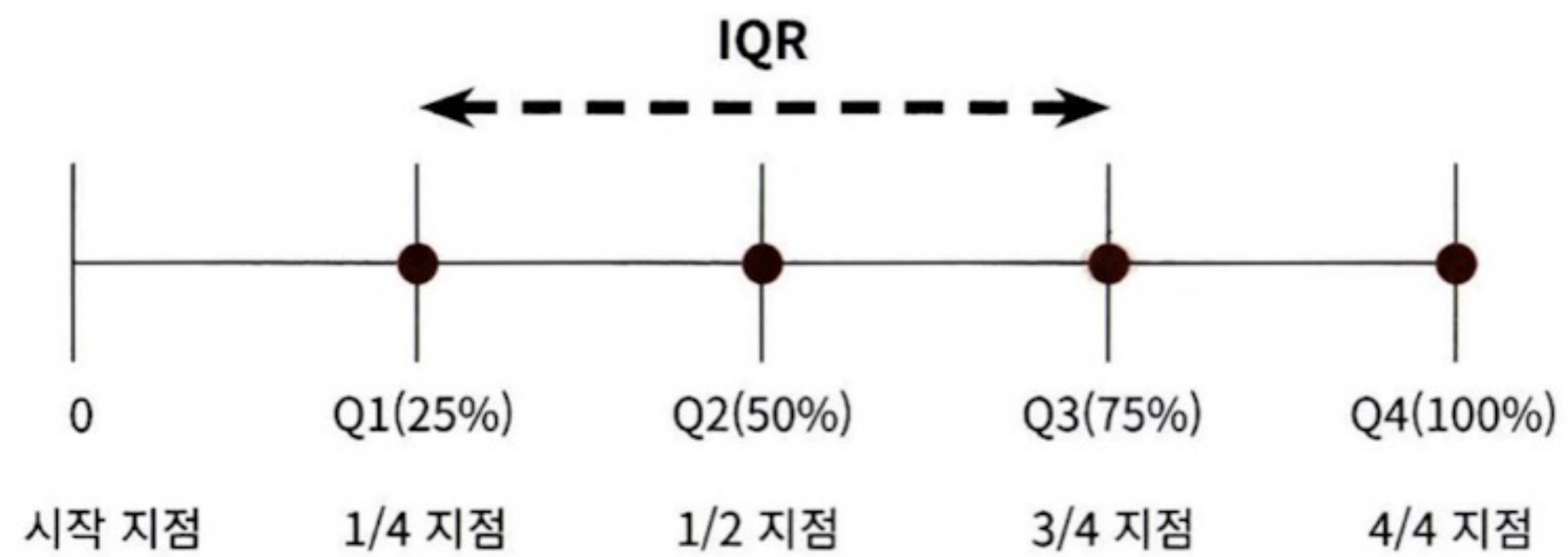
=> **박스 플롯 방식으로** 시각화

이상치 데이터 제거 후 모델 학습/예측/평가

사분위 : 전체 데이터를 값이 **높은 순으로 정렬**한 후, **$\frac{1}{4}$ 씩 분할**하는 방식

< IQR >

- Q1: ~25%
- Q3: ~75%

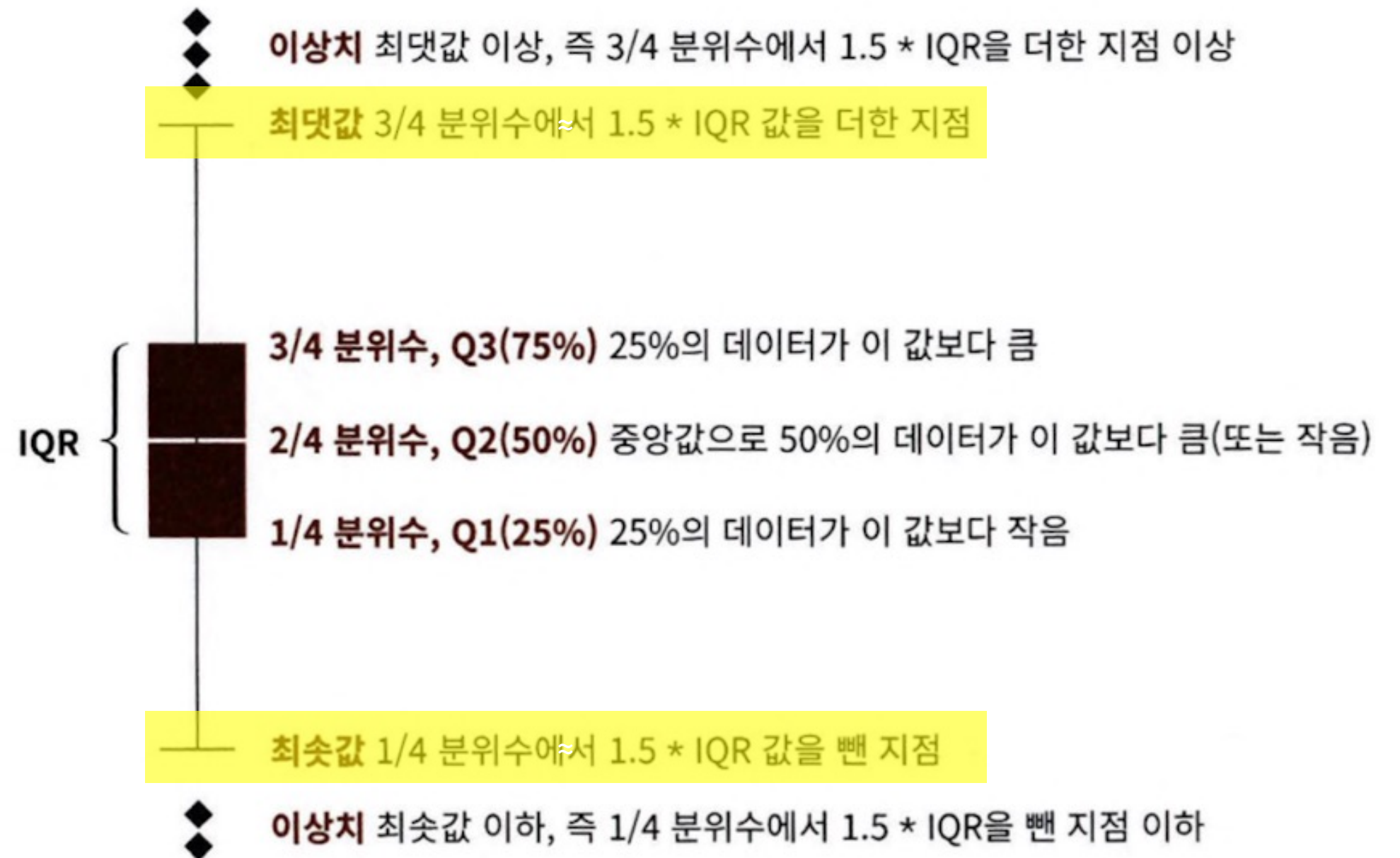


이상치 데이터 제거 후 모델 학습/예측/평가

$IQR \times 1.5$ 로 범위 생성 -> 최댓값, 최솟값 결정 * 1.5는 변할 수 있는 숫자

- 최댓값: $Q3 + IQR \times 1.5$
- 최솟값: $Q1 - IQR \times 1.5$

=> 범위를 벗어난 값을 이상치 데이터로 간주



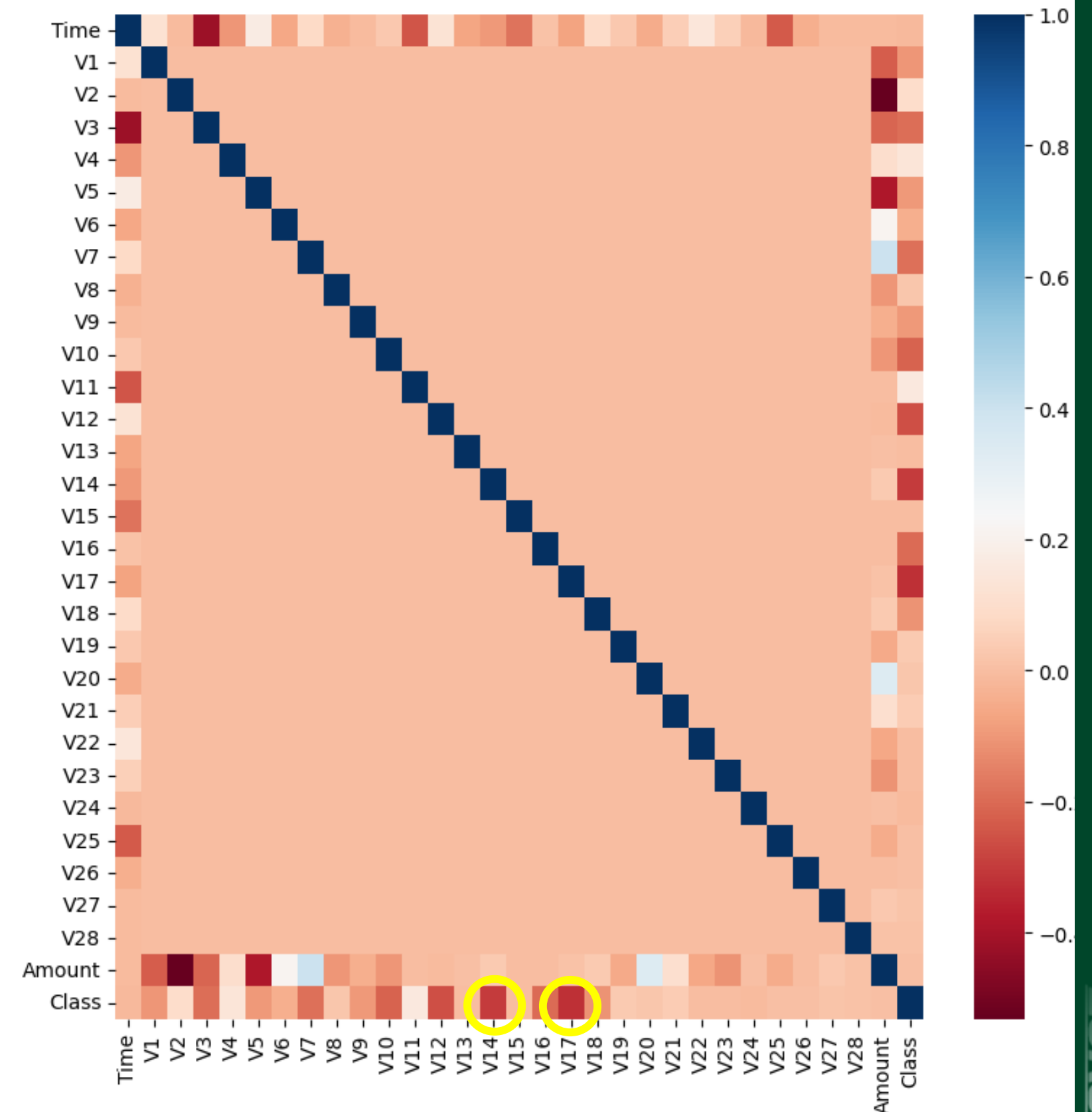
이상치 데이터 제거 후 모델 학습/예측/평가

레이블과 상관성이 높은 피처 위주로 이상치 검출!

```
import seaborn as sns

plt.figure(figsize=(9,9))
corr=card_df.corr()
sns.heatmap(corr,cmap='RdBu')
```

- 양의 상관관계가 높을수록 진한 파란색
- 음의 상관관계가 높을수록 진한 빨간색
- 'Class' 와 음의 상관관계가 가장 높은 피처: V14, V17



이상치 데이터 제거 후 모델 학습/예측/평가

get_outlier()

- 넘파이의 percentile()을 이용해 1/4 분위와 3/4 분위기를 구한 뒤 IQR 계산
- 이상치가 있는 Index를 반환

```
import numpy as np

def get_outlier(df=None, column=None, weight=1.5):
    # fraud에 해당하는 column 데이터만 추출, 1/4 분위와 3/4 분위 지점을 np.percentile로 구함
    fraud=df[df['Class']==1][column]
    quantile_25=np.percentile(fraud.values,25)
    quantile_75=np.percentile(fraud.values,75)

    # IQR을 구하고 IQR에 1.5를 곱해 최댓값과 최솟값 지점 구함
    iqr=quantile_75 - quantile_25
    iqr_weight=iqr*weight
    lowest_val=quantile_25-iqr_weight
    highest_val=quantile_75+iqr_weight
    # 최댓값보다 크거나, 최솟값보다 작은 값을 이상치 데이터로 설정하고 DataFrame index 반환
    outlier_index=fraud[(fraud < lowest_val)| (fraud > highest_val)].index
    return outlier_index
```

이상치 데이터 제거 후 모델 학습/예측/평가

이상치 삭제

```
# get_processed_df()를 로그 변환 후 V14 피처의 이상치 데이터를 삭제하는 로직으로 변경
def get_preprocessed_df(df=None):
    df_copy=df.copy()
    amount_n=np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    # 이상치 데이터 삭제하는 로직 추가
    outlier_index=get_outlier(df=df_copy, column='V14', weight=1.5)
    df_copy.drop(outlier_index, axis=0, inplace=True)
    return df_copy
```

이상치 데이터 제거 후 모델 학습/예측/평가

로지스틱 회귀 & LightGBM 학습 성능 비교

로지스틱 회귀 예측 성능

오차 행렬

```
[[85280    15]
 [    48    98]]
```

정확도: 0.9993, 정밀도: 0.8673, 재현율: ≈ 0.6712 , F1: 0.7568, AUC: 0.9725

LightGBM 예측 성능

오차 행렬

```
[[85290     5]
 [    25   121]]
```

정확도: 0.9996, 정밀도: 0.9603, 재현율: ≈ 0.8288 , F1: 0.8897, AUC: 0.9780

=> 재현율이 크게 상승

SMOTE 오버샘플링 적용 후 모델 학습/예측/평가

`fit_resample()`

: 학습 피처/레이블 데이터를 증식

```
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
```



SMOTE 적용 전 학습용 피처/레이블 데이터 세트: (199362, 29) (199362,)

SMOTE 적용 후 학습용 피처/레이블 데이터 세트: (398040, 29) (398040,)

SMOTE 적용 후 레이블 값 분포:

Class

0 199020

1 199020

SMOTE 오버샘플링 적용 후 모델 학습/예측/평가

SMOTE 적용 후 로지스틱 회귀 모델 학습

```
lr_clf=LogisticRegression(max_iter=1000)
# ftr_train과 tgt_train 인자값이 SMOTE 증식된 X_train_over와 y_train_over로 변경됨에 유의
get_model_train_eval(lr_clf,ftr_train=X_train_over,ftr_test=X_test,tgt_train=y_train_over,tgt_test=y_test)
```

오차 행렬

```
[[82933  2362]
 [   11   135]]
```

정확도: 0.9722, 정밀도: 0.0541, 재현율: 0.9247, F1: 0.1022, AUC: 0.9736

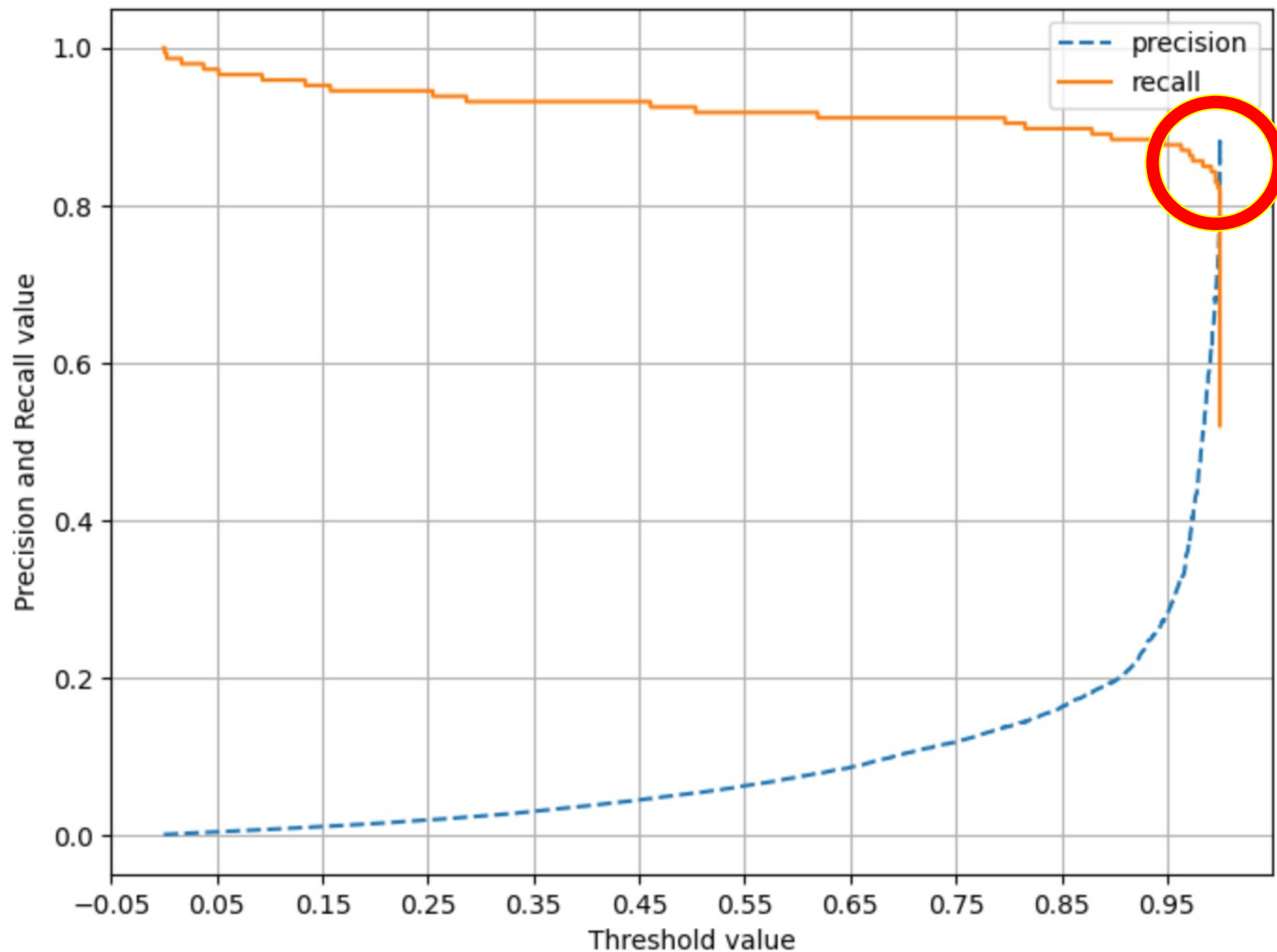
-> 재현율 크게 상승 but 정밀도 급격하게 저하

Why?

=> 오버샘플링으로 인해 원본 데이터보다 매우 많은 'Class' = 1 데이터 학습 때문

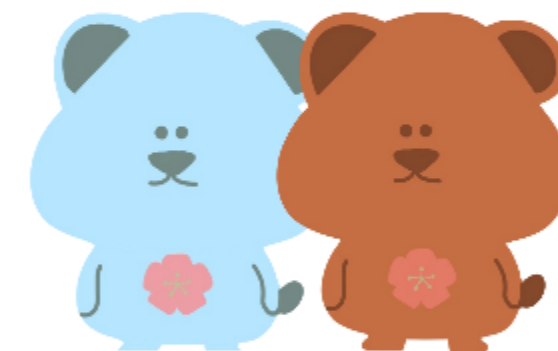
SMOTE 오버샘플링 적용 후 모델 학습/예측/평가

정밀도 / 재현율 trade-off



SMOTE 적용 시)
재현율은 높아지지만 정밀도가 낮아짐
⇒ 정밀도보다 재현율을 높일 때
SMOTE를 사용해야함!

Beginner Friendly CatBoost with Optuna 실습



#00 CatBoost의 특징

- 대부분이 범주형 변수로 이루어진 데이터셋에서 예측 성능이 우수
- Boosting 기반의 모델

CatBoost 알고리즘 특징

1) 특이한 Categorical feature 처리 : 알아서 진행

- ➔ 범주형 변수를 One-hot encoding, Label Encoding 등 encoding 작업을 하지 않고도 그대로 모델의 input으로 사용될 수 있음
- ➔ Ordered Target Encoding : 과거의 데이터를 이용해 현재의 데이터를 인코딩 (data leakage 방지)

2) Ordered Boosting

- Random permutation (train set의 row 순서를 랜덤으로 섞음) -> 오버피팅 방지

#01 데이터 준비

데이터셋 : 캐글 심장병 발병 데이터셋
(<https://www.kaggle.com/fedesoriano/heart-failure-prediction?resource=download>)

≡

kaggle

+

Create

🏠

Home

🏆

Competitions

📁

Datasets

🔗

Models

⌂

Code

💬

Discussions

📖

Learn

⌵

More

📁

Your Work

▼

VIEWED

🖼️

Heart Failure Predic...

🖼️

Credit Card Fraud D...

🔍

Search

👤

FEDESORIANO · UPDATED 4 YEARS AGO

▲

2807

⌂

Code

⬇️

Download

🟡

⋮

🖼️

Heart Failure Prediction Dataset

11 clinical features for predicting heart disease events.

Data Card

Code (1329)

Discussion (28)

Suggestions (0)

About Dataset

Similar Datasets

- Hepatitis C Dataset: [LINK](#)
- Body Fat Prediction Dataset: [LINK](#)
- Cirrhosis Prediction Dataset: [LINK](#)
- Stroke Prediction Dataset: [LINK](#)
- Stellar Classification Dataset - SDSS17: [LINK](#)

Usability ⓘ

10.00

License

Database: Open Database, Cont...

Expected update frequency

Never

Tags

Health

Health Conditions

🖼️

Heart Failure Prediction Dataset

EWHA
EURO

#01 데이터 준비 : 필요한 모듈 임포트

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import KFold, cross_val_score, RepeatedStratifiedKFold, StratifiedKFold
7 from sklearn.impute import SimpleImputer
8 from sklearn.pipeline import Pipeline
9 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
10 from sklearn.preprocessing import OneHotEncoder, StandardScaler, PowerTransformer
11 from sklearn.compose import ColumnTransformer
12 from sklearn.pipeline import Pipeline
13 from sklearn.linear_model import LogisticRegression
14 from sklearn.svm import SVC
15 from sklearn.impute import SimpleImputer
16 from sklearn.dummy import DummyClassifier
17 from imblearn.over_sampling import SMOTE
18
19 from sklearn.ensemble import AdaBoostClassifier
20 from sklearn.ensemble import GradientBoostingClassifier
21 from sklearn.ensemble import RandomForestClassifier
22 from sklearn.ensemble import ExtraTreesClassifier
23 from sklearn.neighbors import KNeighborsClassifier
24
25 import optuna
26 from xgboost import XGBClassifier
27 from lightgbm import LGBMClassifier
28 from catboost import CatBoostClassifier
29
30 from sklearn.pipeline import make_pipeline
31 from sklearn.pipeline import Pipeline
32 from sklearn.compose import make_column_transformer
33
```

```
34 from sklearn.model_selection import KFold, cross_val_predict, train_test_split, GridSearchCV, cross_val_score
35 from sklearn.metrics import accuracy_score, classification_report
36
37 #importing plotly and cufflinks in offline mode
38 import cufflinks as cf
39 cf.go_offline(connected=True)
40 cf.set_config_file(offline=True, world_readable=True)
41
42 import plotly
43 import plotly.express as px
44 import plotly.graph_objs as go
45 import plotly.io as pio
46 pio.renderers.default = "colab"
47
48 from plotly.offline import iplot
49 from plotly.subplots import make_subplots
50 import plotly.figure_factory as ff
51
52 import missingno as msno
53
54 import warnings
55 warnings.filterwarnings("ignore")
```

#01 데이터 준비

- 데이터프레임 불러오기

```
1 from google.colab import drive
2 import pandas as pd
3
4 pd.set_option('display.max_columns', 100)
5 pd.set_option('display.max_rows', 900)
6 pd.set_option('display.max_colwidth', 200)
7
8 # 구글 드라이브 마운트
9 drive.mount('/content/drive')
10
11 file_path = '/content/drive/My Drive/heart.csv'
12 df = pd.read_csv(file_path)
13
14 # 데이터프레임 확인
15 df.head()
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

-info 확인, 중복 여부 확인

```
[5] 1 df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
Column Non-Null Count Dtype

0 Age 918 non-null int64
1 Sex 918 non-null object
2 ChestPainType 918 non-null object
3 RestingBP 918 non-null int64
4 Cholesterol 918 non-null int64
5 FastingBS 918 non-null int64
6 RestingECG 918 non-null object
7 MaxHR 918 non-null int64
8 ExerciseAngina 918 non-null object
9 Oldpeak 918 non-null float64
10 ST_Slope 918 non-null object
11 HeartDisease 918 non-null int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB

```
[6] 1 df.duplicated().sum()
```

0

#01 데이터 준비

- missing values 확인 -> X

```
1 def missing(df):  
2     missing_number = df.isnull().sum().sort_values(ascending=False)  
3     missing_percent = (df.isnull().sum()/df.isnull().count()).sort_values(ascending=False)  
4     missing_values = pd.concat([missing_number, missing_percent], axis=1, keys=['Missing_Number', 'Missing_Percent'])  
5     return missing_values  
6  
7 missing(df)
```

	Missing_Number	Missing_Percent
Age	0	0.0
Sex	0	0.0
ChestPainType	0	0.0
RestingBP	0	0.0
Cholesterol	0	0.0
FastingBS	0	0.0
RestingECG	0	0.0
MaxHR	0	0.0
ExerciseAngina	0	0.0
Oldpeak	0	0.0
ST_Slope	0	0.0
HeartDisease	0	0.0

#01 데이터 준비

```
[8] 1 numerical= df.drop(['HeartDisease'], axis=1).select_dtypes('number').columns
    2
    3 categorical = df.select_dtypes('object').columns
    4
    5 print(f'Numerical Columns: {df[numerical].columns}')
    6 print('\n')
    7 print(f'Categorical Columns: {df[categorical].columns}')
```

```
Numerical Columns: Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak'], dtype='object')
```

```
Categorical Columns: Index(['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope'], dtype='object')
```

```
1 df[categorical].nunique()
```

	0
Sex	2
ChestPainType	4
RestingECG	3
ExerciseAngina	2
ST_Slope	3

dtype: int64

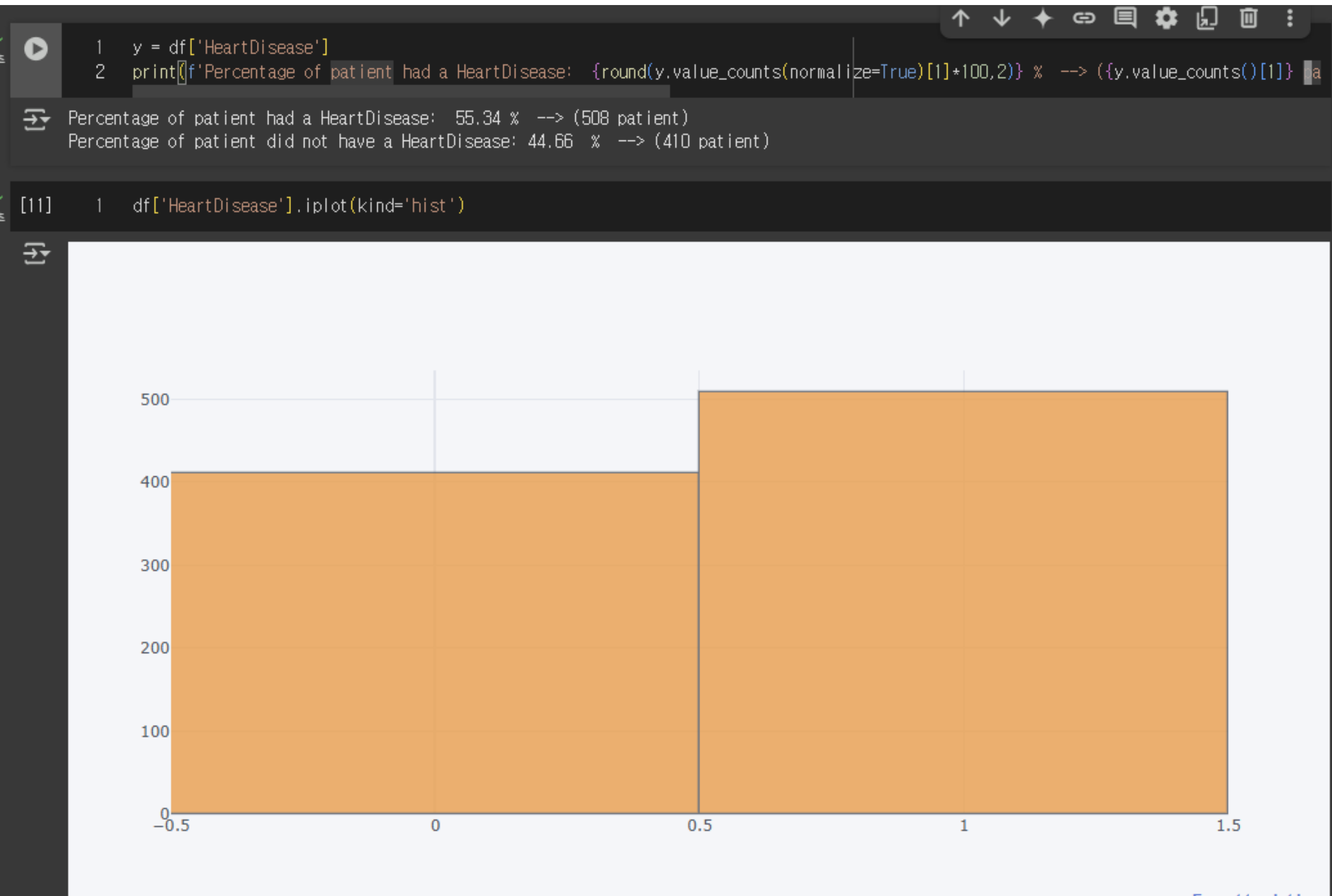
1. 숫자형 변수와 범주형 변수 분리
2. 각각의 컬럼 출력
3. 각 범주형 변수의 고유값 개수 확인
 - Sex : 고유값 2개
 - ChestPainType : 고유값 4개
 - RestingECG : 고유값 3개
 - ExerciseAngina : 고유값 2개
 - ST_Slope : 고유값 3개

No zero variance : 하나의 값만 가지는 변수 X

No extremely high variance (과도한 분산)

: 너무 많은 다양한 값이 있는 변수 X

#02 EDA – Target variable



- HeartDisease 열을 y 변수에 저장
- `value_counts(normalize=True)`로 비율을 계산하고, 백분율로 변환
- 심장병이 있는 환자의 비율과 수, 심장병이 없는 환자의 비율과 수를 나타냄
- Plotly 시각화 기능을 사용해 히스토그램을 그림
- `iplot()` : pandas와 plotly 라이브러리를 활용해 데이터를 interactive(통합) 방식으로 시각화

#02 EDA – Features

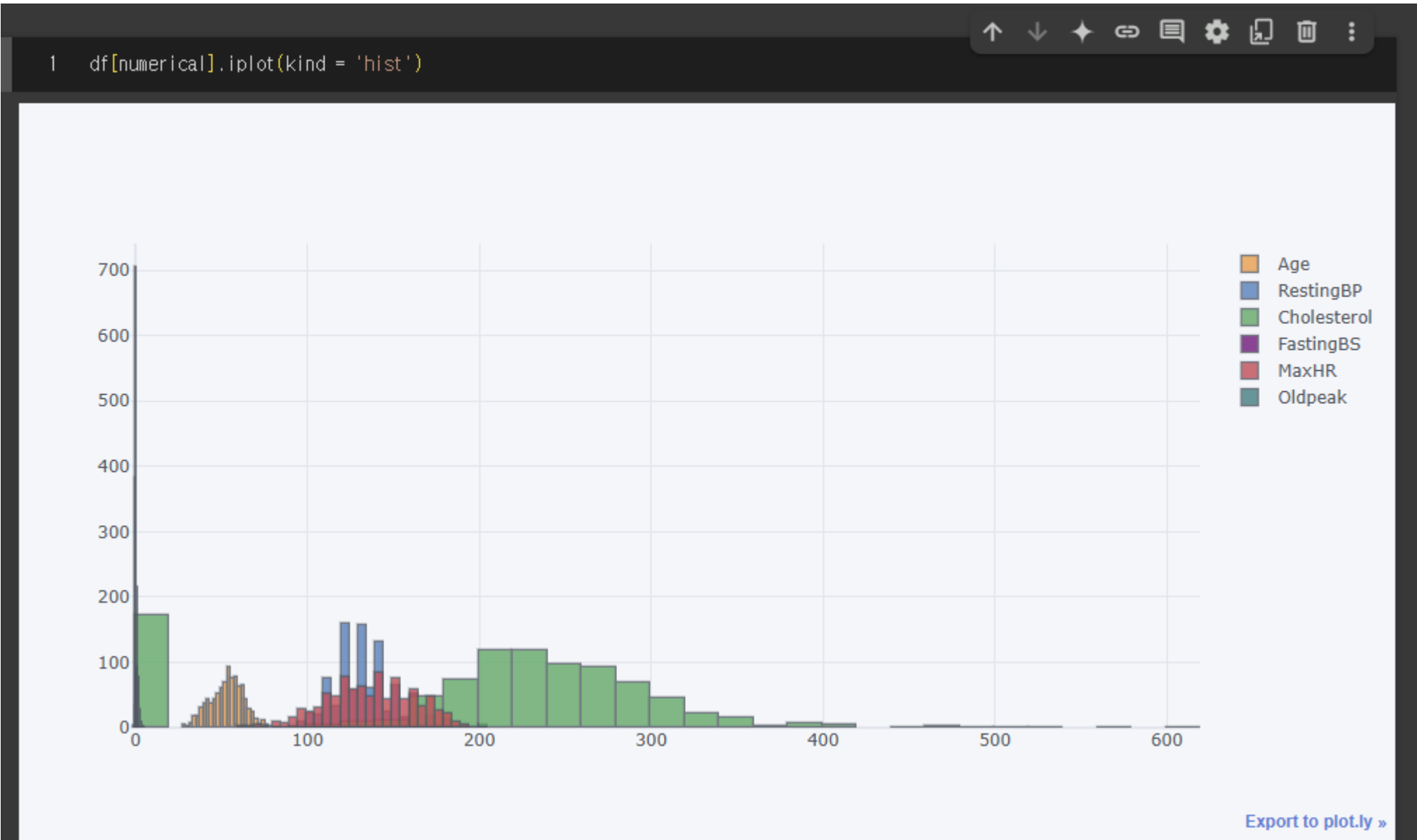
수치형 변수 (Numerical Features)

```
1 df[numerical].describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000

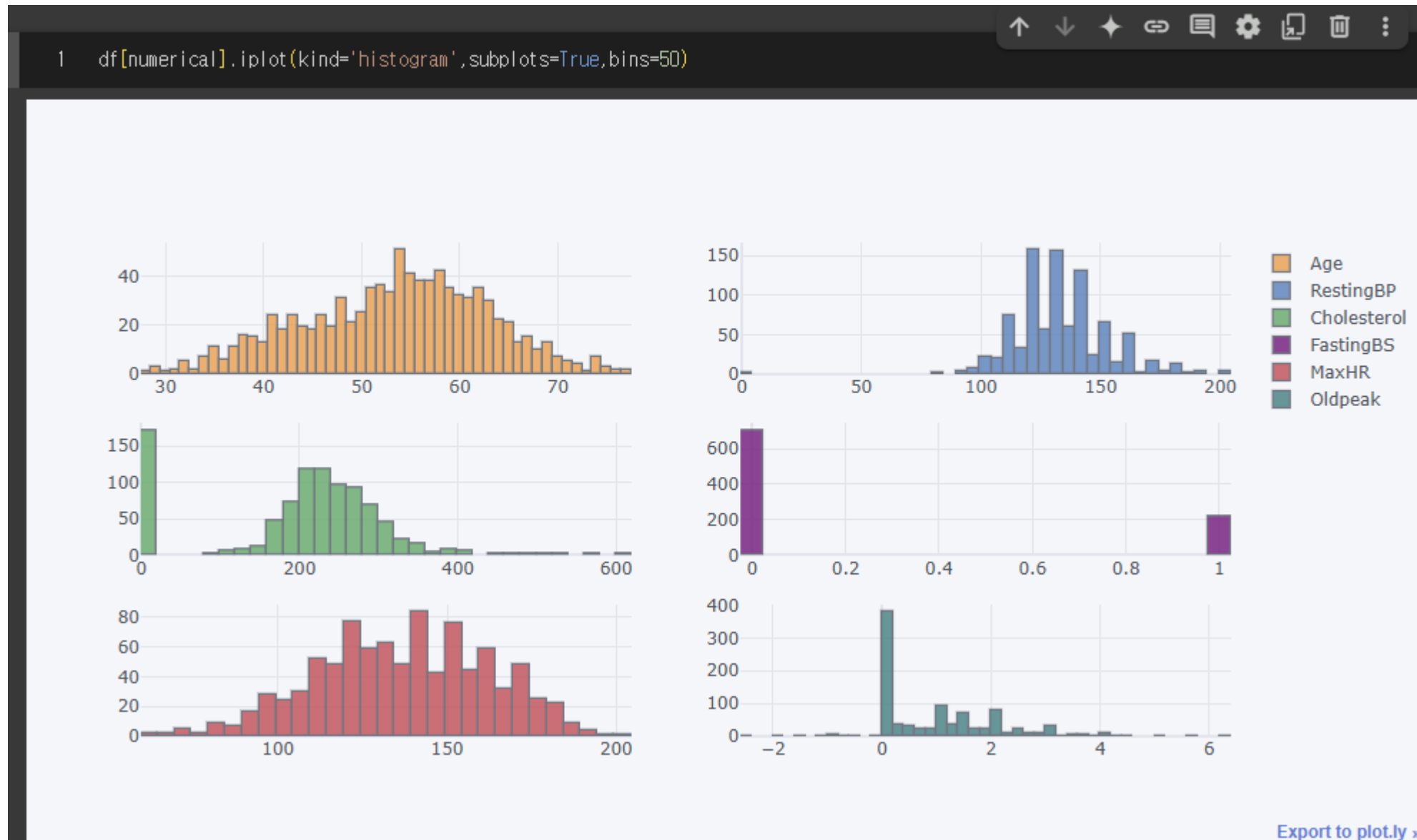
- Numerical 변수에 해당하는 숫자형 열 선택, 선택된 열들에 대한 기술 통계를 계산하여 출력

- Numerical 변수에 해당하는 숫자형 열들을 선택 후, 히스토그램을 그림
- 빈도수와 분포의 형태 확인



#02 EDA – Features

수치형 변수 (Numerical Features)



- Numerical 변수에 해당하는 숫자형 열들을 선택 후, 히스토그램을 그림
 - 빈도수와 분포의 형태 확인
 - 숫자형 변수에 대해 각각의 히스토그램을 그림 (subplot)
 - 구간 수 : 50
-
- 데이터의 왜도를 계산
 - Skew_limit : 왜도의 임계값(threshold)
 - Skew_cols : 왜도가 0.75 이상 ->비대칭적

```
1 skew_limit = 0.75
2 skew_vals = df[numerical].drop('FastingBS', axis=1).skew()
3 skew_cols= skew_vals[abs(skew_vals)> skew_limit].sort_values(ascending=False)
4 skew_cols
```

0

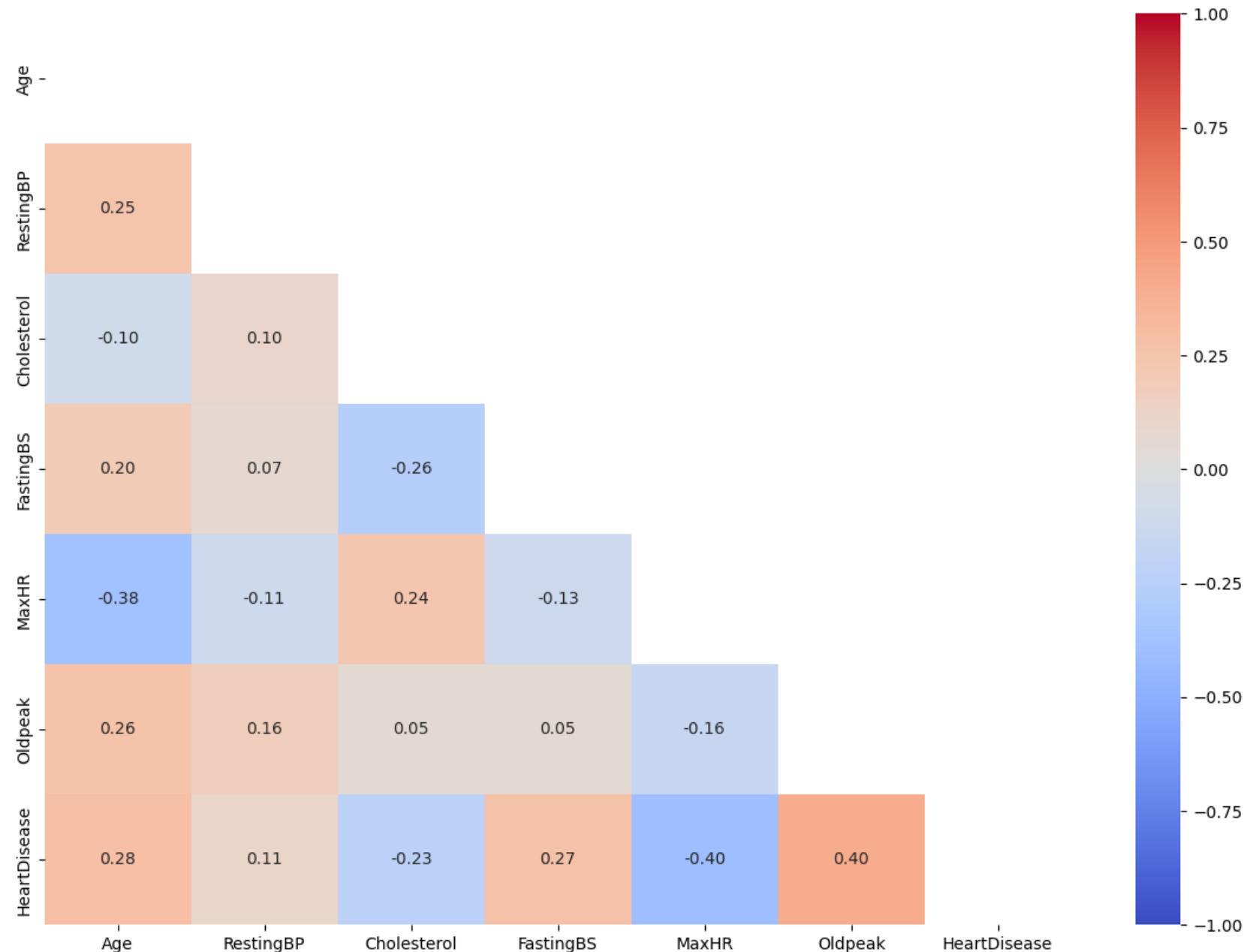
Oldpeak 1.022872

dtype: float64

#02 EDA – Features

수치형 변수 (Numerical Features)

```
1 numerical1= df.select_dtypes('number').columns
2
3
4 matrix = np.triu(df[numerical1].corr())
5 fig, ax = plt.subplots(figsize=(14,10))
6 sns.heatmap(df[numerical1].corr(), annot=True, fmt= '.2f', vmin=-1, vmax=1, center=0, cmap='coolwarm', mask=matrix, ax=ax);
```



- 상관 행렬 (correlation matrix)을 계산
- Np.triu() : upper triangle matrix 추출, 대각선 아래에 있는 값을 모두 제로로 설정
- 히트맵 (heatmap)으로 시각화 : 상관 관계를 시각적으로 확인 가능
- 상관 행렬 -> 수치형 특성과 타겟 변수 (heart disease)간에 약한 수준의 상관관계 존재
- Oldpeak 변수(우울증 관련 수치)가 심장병과 양의 상관관계 존재
- MaxHR(최대 심박수) 가 심장병과 음의 상관관계
- 콜레스테롤 수치가 심장병과 음의 상관관계

#02 EDA – Features

범주형 변수 (Categorical Features)

```
1 df[categorical].head()
```

	Sex	ChestPainType	RestingECG	ExerciseAngina	ST_Slope
0	M	ATA	Normal	N	Up
1	F	NAP	Normal	N	Flat
2	M	ATA	ST	N	Up
3	F	ASY	Normal	Y	Flat
4	M	NAP	Normal	N	Up

- Df[categorical].head()으로 범주형 변수들의 처음 5개 데이터 확인

```
1 print(f'A female person has a probability of {round(df[df["Sex"]=="F"]["HeartDisease"].mean()*100,2)} % have a HeartDisease')
2 print()
3
4 print(f'A male person has a probability of {round(df[df["Sex"]=="M"]["HeartDisease"].mean()*100,2)} % have a HeartDisease')
5 print()
```

A female person has a probability of 25.91 % have a HeartDisease

A male person has a probability of 63.17 % have a HeartDisease

성별에 따라 심장병을 앓을 확률을 계산하고 출력

[결과]

여성의 심장병 확률 : 25.91%

남성의 심장병 확률 : 63.17%

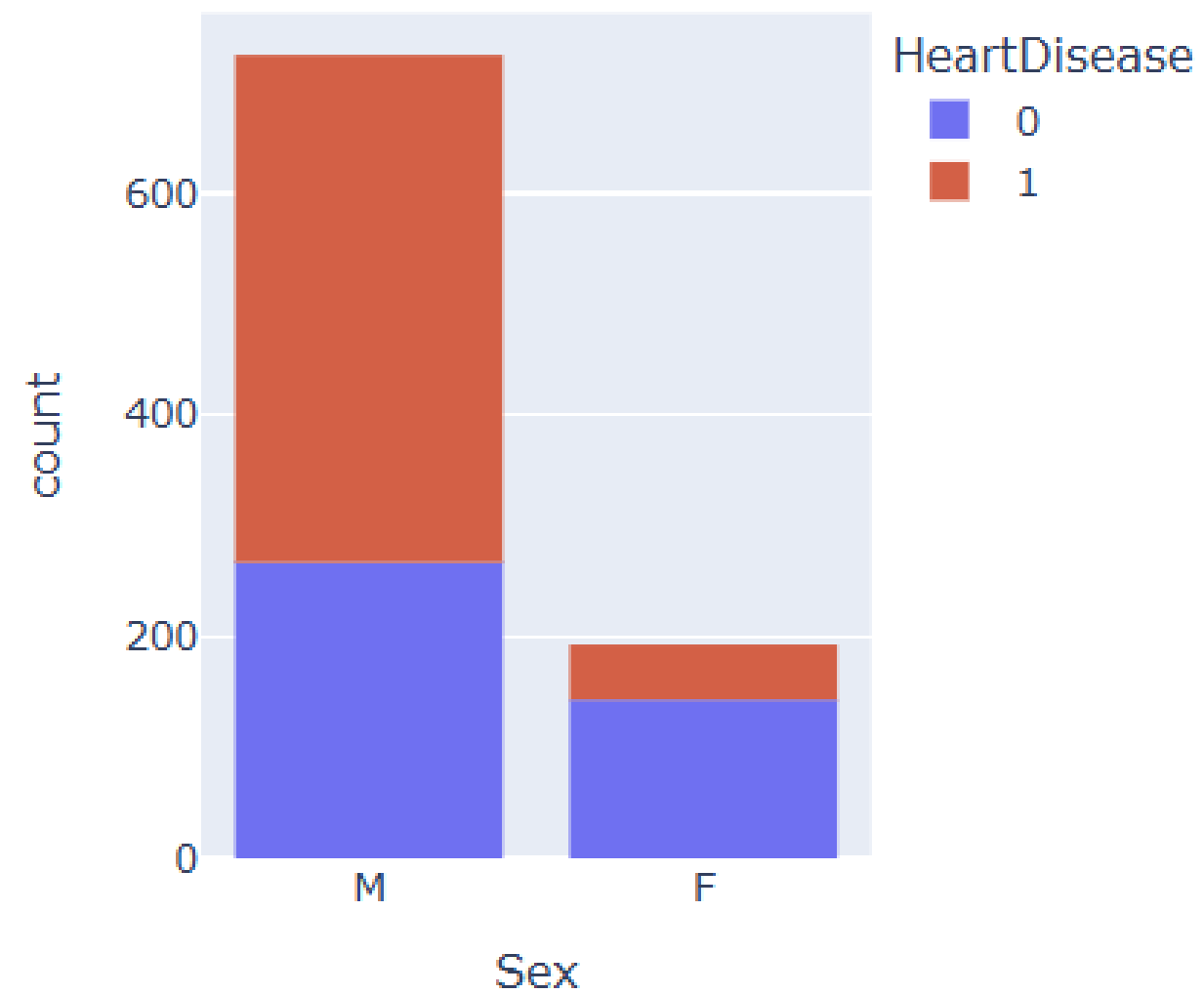
➔ 여성에 비해 남성의 심장병 비율이 훨씬 높음

➔ 성별에 따른 심장병 발생률 차이 0

#02 EDA – Features

범주형 변수 (Categorical Features)

```
1 fig = px.histogram(df, x="Sex", color="HeartDisease",width=400, height=400)
2 fig.show()
```



- px.histogram() : plotly.express에서 제공하는 히스토그램을 생성하는 함수
- 성별에 따라 심장병 여부를 시각화

#02 EDA – Features

범주형 변수 (Categorical Features)

```
1 df.groupby('ChestPainType')['HeartDisease'].mean().sort_values(ascending=False)
```

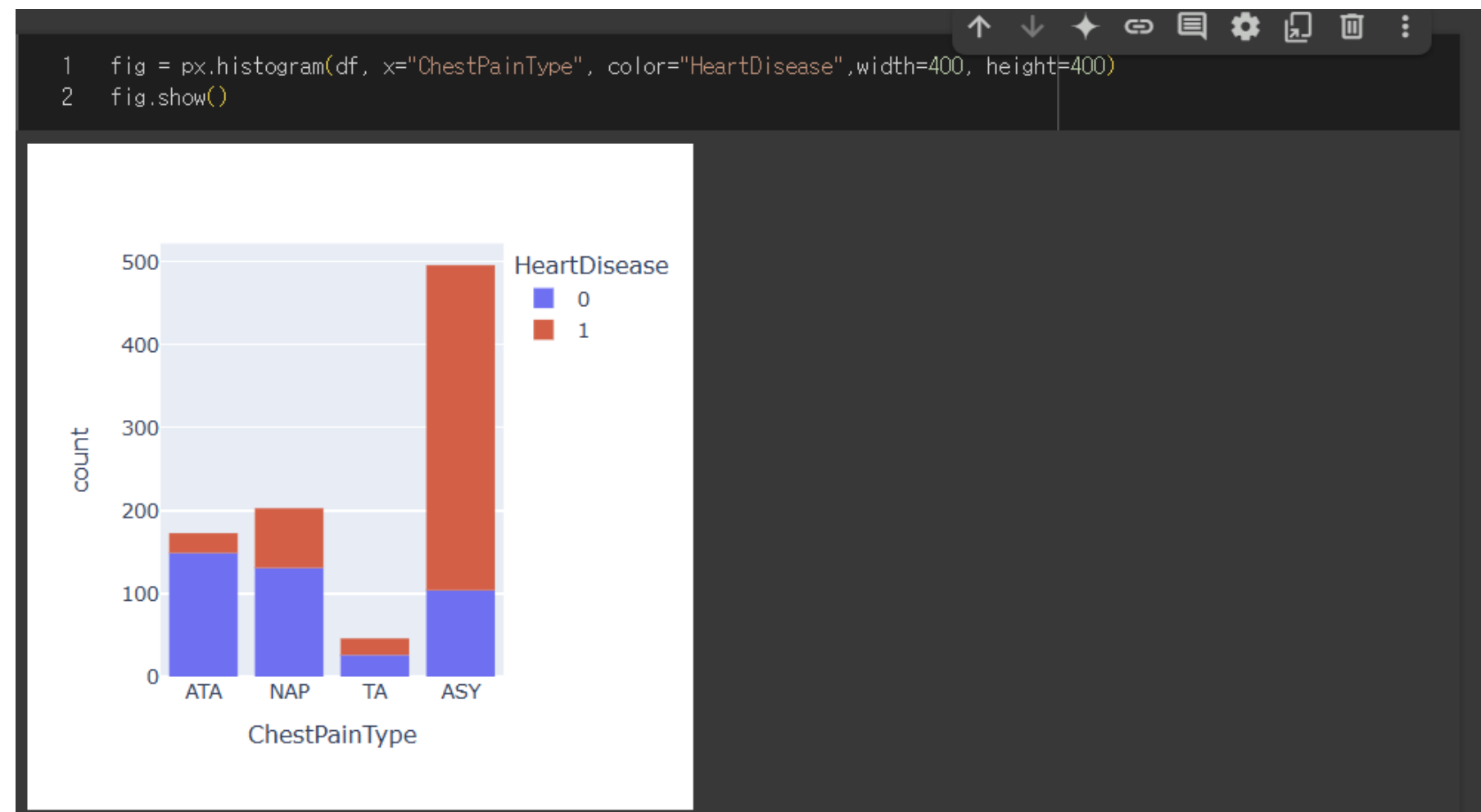
HeartDisease	
ChestPainType	
ASY	0.790323
TA	0.434783
NAP	0.354680
ATA	0.138728

dtype: float64

- ChestPainType(흉통 유형)별로 심장병 유무의 평균값을 내림차순으로 정렬
 - ASY(무증상 흉통)는 심장병 발생 확률 높음, ATA(전형적인 흉통)는 심장병 발생 확률 가장 낮음
- > 흉통이 있지만 증상이 없는 경우 심장병이 있을 확률이 매우 높다

ChestPainType(흉통 유형)과 HeartDisease 간의 관계를 히스토그램으로 시각화

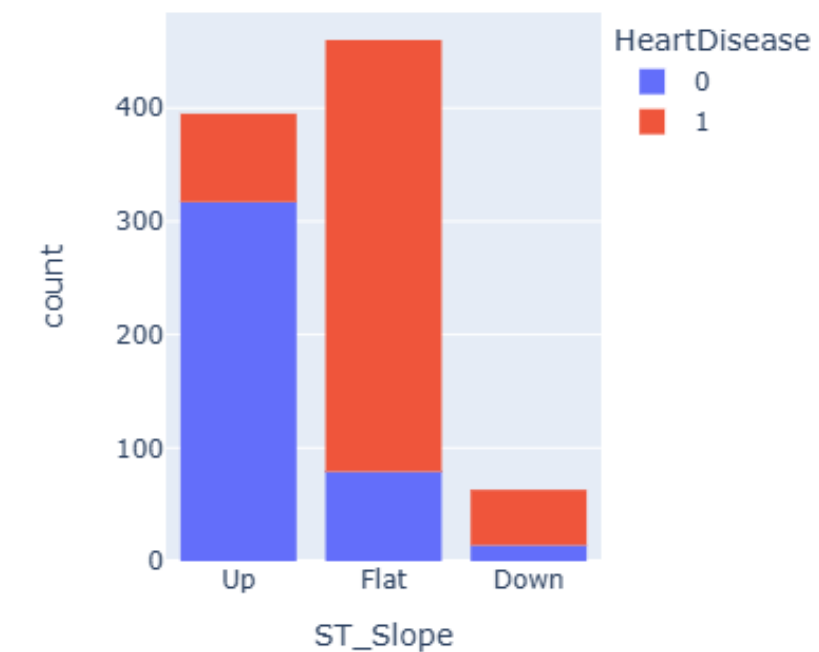
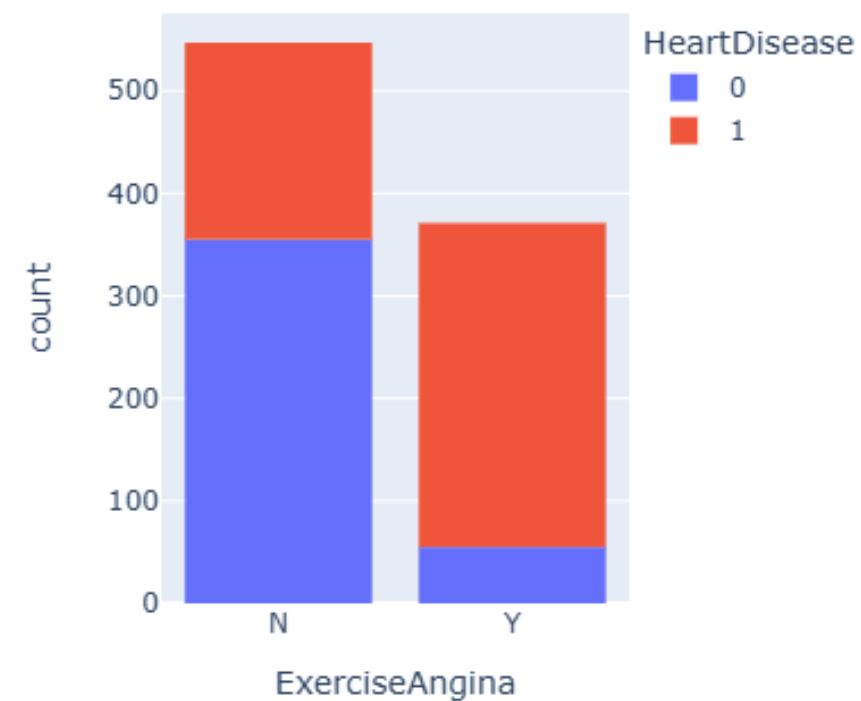
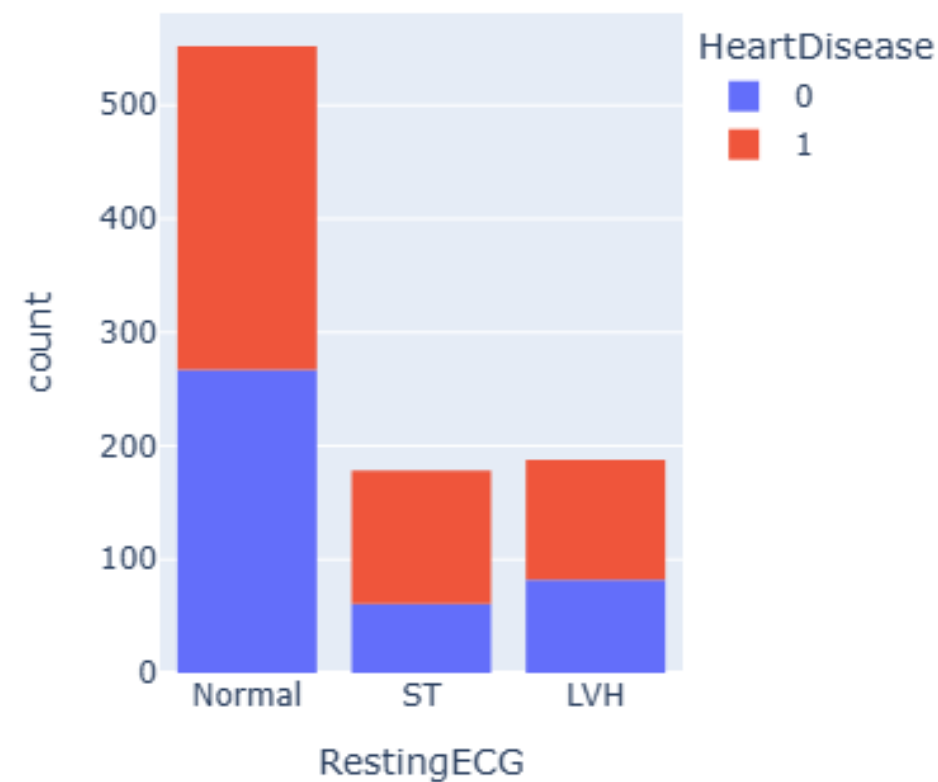
- ASY를 가진 사람이 심장병을 앓을 확률은 ATA (비전형적 흉통)를 가진 사람에 비해 약 6배 더 높다.
- > 무증상 흉통이 심장병의 중요한 위험 요소임을 보여줌



#02 EDA – Features

범주형 변수 (Categorical Features)

- RestingECG(휴식 시 심전도) 유형별 심장병 발생 비율 계산, 시각화
- ExerciseAngina(운동 유발 흉통) 유형별 심장병 발생 비율 계산, 시각화
- ST_Slope 열 기준 그룹화, 유형별 심장병 발생 비율 계산, 시각화



#02 EDA – 인사이드

- 타겟 변수인 HeartDisease는 상당히 균형을 이룸
- 수치형 변수와 타겟 변수 간 약한 상관관계
- Oldpeak(우울증 관련 수치)는 심장병과 positive correlation
- 최대심박수는 심장병과 negative correlation
- 콜레스테롤은 심장병과 negative correlation
- 남성이 여성보다 심장병에 걸릴 확률이 더 높음
- 흉통 유형 (ASY, ATA 등)에 따라 심장병 발생 확률 차이 존재
- RestingECG (휴식 심전도)는 큰 차이 X
- ST_Slope도 심장병 발생 확률에 영향 0

#03 model selection

Dummy Classifier 모델 : 기본 모델

Logistic Regression

Linear Discriminant Analysis(ADA, 선형 판별 분석)

K-최근접 이웃 (Kneighbors)

서포트 벡터 머신 (SVM)

-> SCALER 0,X로 실험

앙상블 모델 -> AdaBoost, Random Forest, Gradient Boosting, Extra Trees 적용

XGBoost, LightGBM. CatBoost 모델 살펴보기

#03 model selection

#Baseline Model

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8
9 ohe= OneHotEncoder()
10 ct= make_column_transformer((ohe,categorical),remainder='passthrough')
11
12
13 model = DummyClassifier(strategy='constant', constant=1)
14 pipe = make_pipeline(ct, model)
15 pipe.fit(X_train, y_train)
16 y_pred = pipe.predict(X_test)
17 accuracy.append(round(accuracy_score(y_test, y_pred),4))
18 print(f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')
19
20 model_names = ['DummyClassifier']
21 dummy_result_df = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
22 dummy_result_df
```

model : DummyClassifier(constant=1, strategy='constant') and accuracy score is : 0.5942

	Accuracy
DummyClassifier	0.5942

Dummy Classifier 사용

- 훈련/테스트 데이터 분리 (70/30)
- One-hot encoding
- Constant =1 -> 항상 1 예측
- make_pipeline(ct,model): 모델을 하나의 파이프라인으로 묶음
- Pipe.fit(X_train, y_train) : 모델 훈련
- Pipe.predict(X_test): 예측 수행
- Accuracy_score(y_test, y_pred) : 모델 정확도 평가

Dummy Classifier 모델의 정확도 : 59.42%

#03 model selection

#Logistic & Linear Discriminant & SVC & KNN

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8
9 ohe= OneHotEncoder()
10 ct= make_column_transformer((ohe,categorical),remainder='passthrough')
11
12
13 lr = LogisticRegression(solver='liblinear')
14 lda= LinearDiscriminantAnalysis()
15 svm = SVC(gamma='scale')
16 knn = KNeighborsClassifier()
17
18 models = [lr,lda,svm,knn]
19
20 for model in models:
21     pipe = make_pipeline(ct, model)
22     pipe.fit(X_train, y_train)
23     y_pred = pipe.predict(X_test)
24     accuracy.append(round(accuracy_score(y_test, y_pred),4))
25     print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')
26
27 model_names = ['Logistic','LinearDiscriminant','SVM','KNeighbors']
28 result_df1 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
29 result_df1
```

Lr -> 로지스틱 회귀 모델
Lda -> 선형 판별 분석 모델
Svm -> 서포트 벡터 a신
Knn -> K-최근접 이웃 모델 정의

여러 분류 모델을 훈련시켜 정확도 계산

- 훈련/테스트 데이터 분리
- One-hot Encoding (ct)

모델 훈련 및 예측

- 파이프라인
- 훈련 데이터로 모델 학습, 테스트 데이터에 대해 예측 수행

- 예측 결과에 대한 정확도 계산 (accuracy)
- 정확도 출력

```
model : LogisticRegression(solver='liblinear') and accuracy score is : 0.8841
model : LinearDiscriminantAnalysis() and accuracy score is : 0.8696
model : SVC() and accuracy score is : 0.7246
model : KNeighborsClassifier() and accuracy score is : 0.7174
```

	Accuracy
Logistic	0.8841
LinearDiscriminant	0.8696
SVM	0.7246
KNeighbors	0.7174

#03 model selection

#Logistic & Linear Discriminant & SVC & KNN with Scaler

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8
9 ohe= OneHotEncoder()
10 s= StandardScaler()
11 ct1= make_column_transformer((ohe,categorical),(s,numerical))
12
13
14 lr = LogisticRegression(solver='liblinear')
15 lda= LinearDiscriminantAnalysis()
16 svm = SVC(gamma='scale')
17 knn = KNeighborsClassifier()
18
19 models = [lr,lda,svm,knn]
20
21 for model in models:
22     pipe = make_pipeline(ct1, model)
23     pipe.fit(X_train, y_train)
24     y_pred = pipe.predict(X_test)
25     accuracy.append(round(accuracy_score(y_test, y_pred),4))
26     print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')
27
28 model_names = ['Logistic_scl','LinearDiscriminant_scl','SVM_scl','KNeighbors_scl']
29 result_df2 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
30 result_df2
```

StandardScaler(s) 적용!

StandardScaler 적용 : 수치형 변수들에 대해 정규화 수행

[결과]
스케일러 적용 후, KNN과 SVM 모델 성능 향상
(정확도: 0.8841)

```
model : LogisticRegression(solver='liblinear') and accuracy score is : 0.8804
model : LinearDiscriminantAnalysis() and accuracy score is : 0.8696
model : SVC() and accuracy score is : 0.8841
model : KNeighborsClassifier() and accuracy score is : 0.8841
```

	Accuracy	
Logistic_scl	0.8804	
LinearDiscriminant_scl	0.8696	
SVM_scl	0.8841	
KNeighbors_scl	0.8841	

#03 model selection

#Ensemble Models

(AdaBoost & Gradient Boosting & Random Forest & Extra Trees)

```
1 accuracy =[]
2 model_names =[]
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8
9 ohe= OneHotEncoder()
10 ct= make_column_transformer((ohe,categorical),remainder='passthrough')
11
12 ada = AdaBoostClassifier(random_state=0)
13 gb = GradientBoostingClassifier(random_state=0)
14 rf = RandomForestClassifier(random_state=0)
15 et= ExtraTreesClassifier(random_state=0)
16
17
18
19 models = [ada,gb,rf,et]
20
21 for model in models:
22     pipe = make_pipeline(ct, model)
23     pipe.fit(X_train, y_train)
24     y_pred = pipe.predict(X_test)
25     accuracy.append(round(accuracy_score(y_test, y_pred),4))
26     print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')
27
28 model_names = ['Ada','Gradient','Random','ExtraTree']
29 result_df3 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
30 result_df3
```

앙상블 모델 정의 : ada, gb, rf, et

각 모델이 비슷한 성능을 보임

앙상블 기법들이 좋은 성능, AdaBoost는 상대적으로 낮은 성능

Random Forest, Extra Trees는 하이퍼 파라미터 튜닝을 통해 더 좋은 성능을 낼 수 있음 (n_estimators, max_depth 등 조절)

```
model : AdaBoostClassifier(random_state=0) and accuracy score is : 0.8732
model : GradientBoostingClassifier(random_state=0) and accuracy score is : 0.8768
model : RandomForestClassifier(random_state=0) and accuracy score is : 0.8877
model : ExtraTreesClassifier(random_state=0) and accuracy score is : 0.8804
```

	Accuracy	
Ada	0.8732	
Gradient	0.8768	
Random	0.8877	
ExtraTree	0.8804	

#03 model selection

#Famous Trio (XGBoost & LightGBM & Catboost)

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8
9 ohe= OneHotEncoder()
10 ct= make_column_transformer((ohe,categorical),remainder='passthrough')
11
12 xgbc = XGBClassifier(random_state=0)
13 lgbmc=LGBMClassifier(random_state=0)
14
15
16 models = [xgbc,lgbmc]
17
18 for model in models:
19     pipe = make_pipeline(ct, model)
20     pipe.fit(X_train, y_train)
21     y_pred = pipe.predict(X_test)
22     accuracy.append(round(accuracy_score(y_test, y_pred),4))
23
24 model_names = ['XGBoost','LightGBM']
25 result_df4 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
26 result_df4
```

모델 정의

Xgbc -> XGBoost 모델 정의 (Gradient Boosting 기반)

Lgbmc -> LightGBM 모델 정의

For model in models -> XGBoost와 LightGBM 모델 순차적 학습, 예측 수행

결과 : XGBoost < LightGBM

(Catboost는 범주형 변수를 자동으로 처리 가능 -> 사전 처리 없이도 좋은 성능 보일 수 0)

Accuracy	
XGBoost	0.8478
LightGBM	0.8732

#03 model selection

#CATBOOST

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 categorical_features_indices = np.where(X.dtypes != float)[0]
8
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
10
11 model = CatBoostClassifier(verbose=False, random_state=0)
12
13 model.fit(X_train, y_train, cat_features=categorical_features_indices, eval_set=(X_test, y_test))
14 y_pred = model.predict(X_test)
15 accuracy.append(round(accuracy_score(y_test, y_pred), 4))
16
17 model_names = ['Catboost_default']
18 result_df5 = pd.DataFrame({'Accuracy': accuracy}, index=model_names)
19 result_df5
```

범주형 변수의 인덱스를 찾아 배열에 저장

모델 훈련
범주형 특상의 인덱스 지정 : CatBoost가
범주형 변수를 자동으로 처리하도록

CatBoost는 범주형 데이터를
자동으로 처리

CatBoost 정확도 : 0.8804

→ 기본 설정으로도 이미 좋은
성능을 보임

→ 더 나은 성능을 위해 모델
하이퍼파라미터 튜닝 :
learning_rate, depth,
iterations 등

Accuracy



Catboost_default

0.8804



#03 model selection

하이퍼 파라미터 튜닝을 위한 라이브러리로, 주어진 목표 함수(정확도)를 최적화하는 파라미터 조합을 찾아냄

#CATBOOST HyperParameter Tuning with Optuna

```
1 def objective(trial):
2     X= df.drop('HeartDisease', axis=1)
3     y= df['HeartDisease']
4     categorical_features_indices = np.where(X.dtypes != float)[0]
5
6     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
7
8     param = {
9         "objective": trial.suggest_categorical("objective", ["Logloss", "CrossEntropy"]),
10        "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.01, 0.1),
11        "depth": trial.suggest_int("depth", 1, 12),
12        "boosting_type": trial.suggest_categorical("boosting_type", ["Ordered", "Plain"]),
13        "bootstrap_type": trial.suggest_categorical(
14            "bootstrap_type", ["Bayesian", "Bernoulli", "MVS"]
15        ),
16        "used_ram_limit": "3gb",
17    }
18
19    if param["bootstrap_type"] == "Bayesian":
20        param["bagging_temperature"] = trial.suggest_float("bagging_temperature", 0, 10)
21    elif param["bootstrap_type"] == "Bernoulli":
22        param["subsample"] = trial.suggest_float("subsample", 0.1, 1)
23
24    cat_cls = CatBoostClassifier(**param)
25
26    cat_cls.fit(X_train, y_train, eval_set=[(X_test, y_test)], cat_features=categorical_features_indices, verbose=0, early_stopping_rounds=100)
27
28    preds = cat_cls.predict(X_test)
29    pred_labels = np.rint(preds)
30    accuracy = accuracy_score(y_test, pred_labels)
31    return accuracy
32
```

Param 딕셔너리에
CatBoostClassifier
하이퍼 파라미터들이
정의됨

Optuna 사용
Objective 함수 : Optuna의
최적화 목적 함수, 하이퍼
파라미터 튜닝을 위해 여러 번
호출

```
32
33
34 if __name__ == "__main__":
35     study = optuna.create_study(direction="maximize")
36     study.optimize(objective, n_trials=50, timeout=600)
37
38     print("Number of finished trials: {}".format(len(study.trials)))
39
40     print("Best trial:")
41     trial = study.best_trial
42
43     print("  Value: {}".format(trial.value))
44
45     print("  Params: ")
46     for key, value in trial.params.items():
47         print("    {}: {}".format(key, value))
```

```
[I 2025-04-03 16:02:04,067] A new study created in memory with name: no-name-f3
[I 2025-04-03 16:02:09,529] Trial 0 finished with value: 0.8840579710144928 and
[I 2025-04-03 16:02:13,085] Trial 1 finished with value: 0.8804347826086957 and
[I 2025-04-03 16:02:16,091] Trial 2 finished with value: 0.8840579710144928 and
[I 2025-04-03 16:02:17,295] Trial 3 finished with value: 0.8840579710144928 and
[I 2025-04-03 16:02:18,883] Trial 4 finished with value: 0.8913043478260869 and
[I 2025-04-03 16:02:21,178] Trial 5 finished with value: 0.894927536231884 and
[I 2025-04-03 16:02:23,301] Trial 6 finished with value: 0.8804347826086957 and
[I 2025-04-03 16:02:26,683] Trial 7 finished with value: 0.8840579710144928 and
[I 2025-04-03 16:02:29,743] Trial 8 finished with value: 0.8876811594202898 and
[I 2025-04-03 16:02:31,155] Trial 9 finished with value: 0.894927536231884 and
```

#03 model selection

#CATBOOST HyperParameter Tuning with Optuna

- Trial.suggest_categorical(“objective”, [“Logloss”, “CrossEntropy”]) : 목표 함수는 Logloss or Cross Entropy로 선택됨
- Trial.suggest_int(“depth”, 1, 12) : 트리의 깊이 depth는 1에서 12 사이의 정수로 제시됨
- 부스팅 방식 (boosting_type)은 Ordered or Plain 으로 선택
- 부트스트랩 방식 (bootstrap_type)은 Bayesia, Bernoulli, MVS 중 하나로 선택됨
- CatBoost 모델 훈련 : CatBoostClassifier를 하이퍼 파라미터 param으로 생성, 조기 종료(early stopping) 설정
- colsample_bylevel : 훈련 속도를 높이는 파라미터
- Depth : 트리의 깊이
- 최적화 결과 출력 (study.best_trial) : 최적의 하이퍼 파라미터와 해당 값들 출력

```
Number of finished trials: 50
Best trial:
  Value: 0.8985507246376812
  Params:
    objective: CrossEntropy
    colsample_bylevel: 0.022622486441817607
    depth: 10
    boosting_type: Plain
    bootstrap_type: MVS
```

#03 model selection

#CATBOOST HyperParameter Tuning with Optuna -> Best Model 적용

```
1 accuracy = []
2 model_names = []
3
4
5 X= df.drop('HeartDisease', axis=1)
6 y= df['HeartDisease']
7 categorical_features_indices = np.where(X.dtypes != float)[0]
8
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
10
11 model = CatBoostClassifier(verbose=False, random_state=0,
12                             objective= 'CrossEntropy',
13                             colsample_bylevel= 0.04292240490294766,
14                             depth= 10,
15                             boosting_type= 'Plain',
16                             bootstrap_type= 'MVS')
17
18 model.fit(X_train, y_train, cat_features=categorical_features_indices, eval_set=(X_test, y_test))
19 y_pred = model.predict(X_test)
20 accuracy.append(round(accuracy_score(y_test, y_pred), 4))
21 print(classification_report(y_test, y_pred))
22
23 model_names = ['Catboost_tuned']
24 result_df6 = pd.DataFrame({'Accuracy': accuracy}, index=model_names)
25 result_df6
```

Model_names : 튜닝된 CatBoost 모델 이름
설정

Precision (정밀도), Recall (재현율), F1-score,
Accuracy (정확도)

[결과]
성능 향상
0.8804 -> 0.9094

	precision	recall	f1-score	support
0	0.88	0.90	0.89	112
1	0.93	0.91	0.92	164
accuracy			0.91	276
macro avg	0.90	0.91	0.91	276
weighted avg	0.91	0.91	0.91	276

Accuracy

Catboost_tuned 0.9094

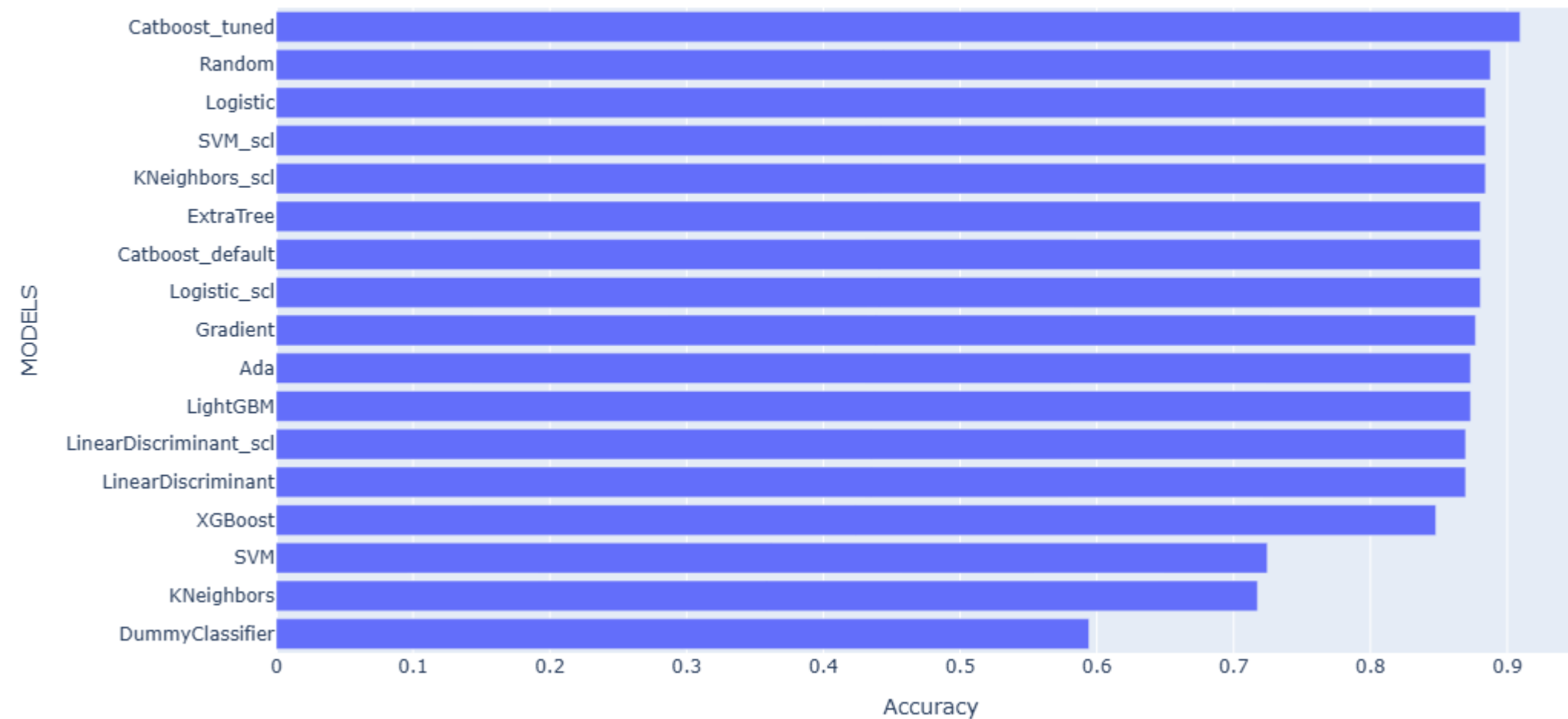
#03 model selection

여러 모델의 성능 비교 시각화

```
[37] 1 result_final = pd.concat([dummy_result_df,result_df1,result_df2,result_df3,result_df4,result_df5,result_df6],axis=0)

1 result_final.sort_values(by=['Accuracy'], ascending=True,inplace=True)
2 fig = px.bar(result_final, x='Accuracy', y=result_final.index,title='Model Comparison',height=600,labels={'index':'MODELS'})
3 fig.show()
```

Model Comparison



Pd.concat -> 결과 결합
정확도 기준으로 모델 정렬
Px.bar을 이용해 막대 그래프 생성

#03 model selection - Conclusion

- EDA 수행
- Target 변수와 feature 분석
- 파이프라인 사용
- 각 모델의 결과를 살펴보고, 주어진 문제에 가장 적합한 모델 선택
- CatBoost 모델
- Optuna를 사용하여 Catboost의 하이퍼 파라미터 튜닝 진행 -> 성능 개선 !
- 특성 중요도 확인

THANK YOU

